

오픈소스SW의 이해

CHAPTER 4

SEMESTER | 2026-2

박성미

목차

Table of Contents



01. 자동 병합하기

02. Branch 병합하기 - 실습

03. 충돌 해결하기

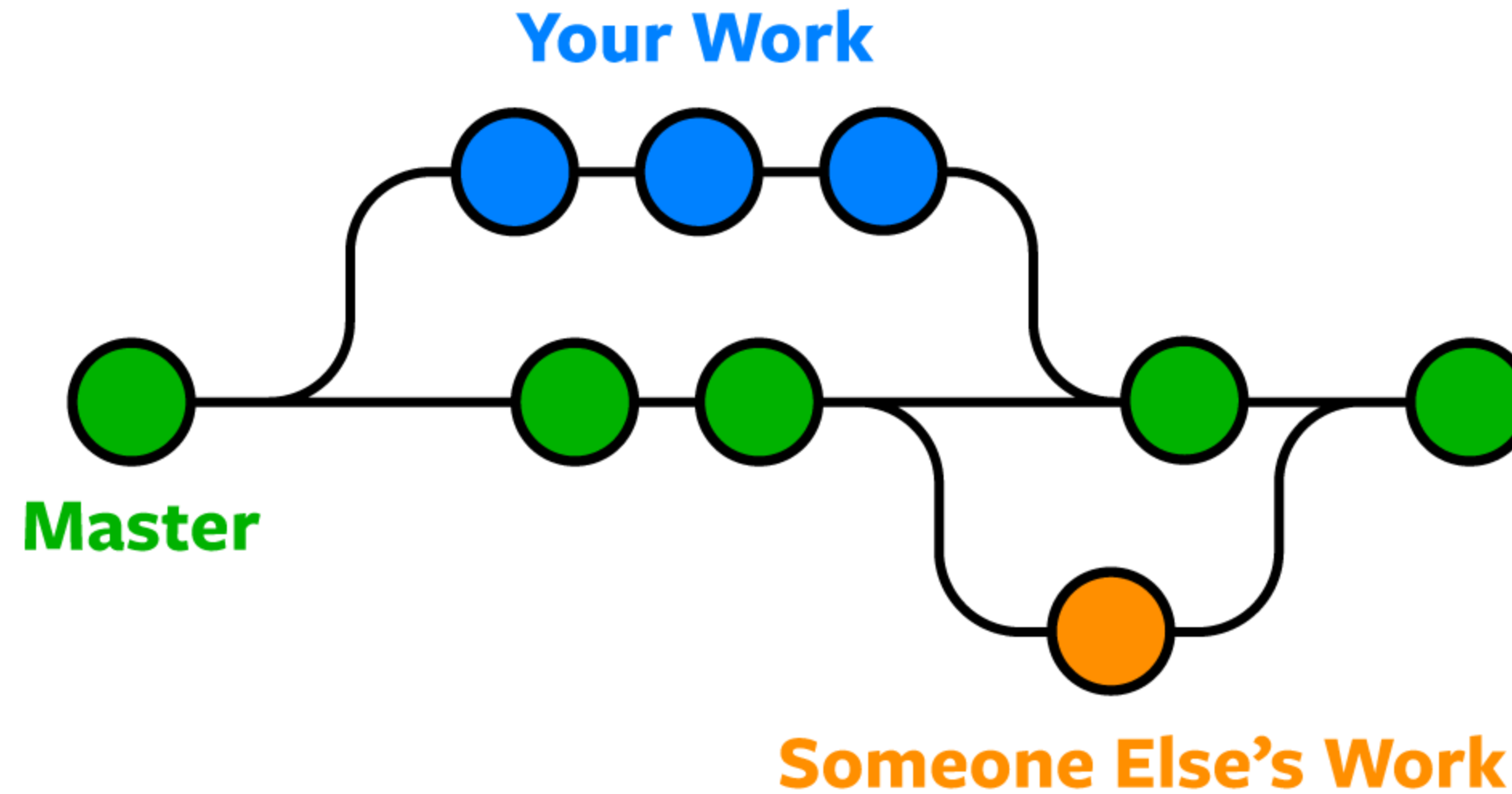
04. Oh My Git



복습

한림대학교

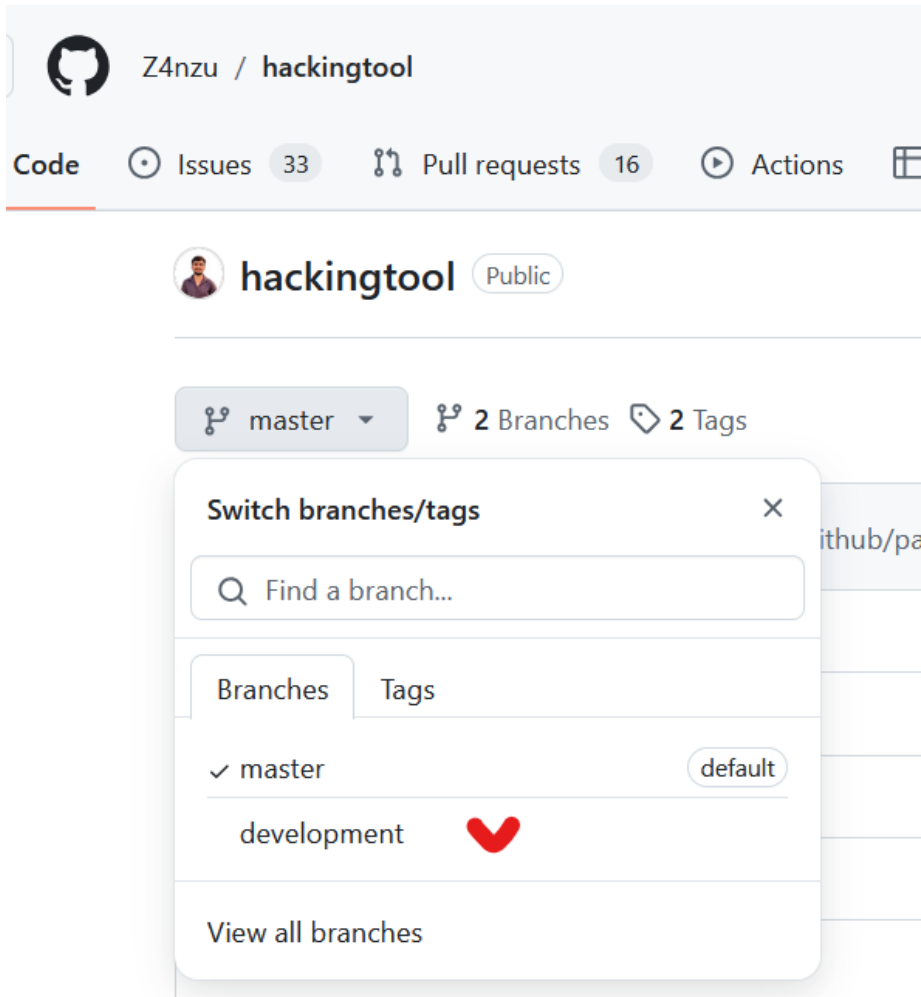
Git Branch 기초



❖ Git Branch란?

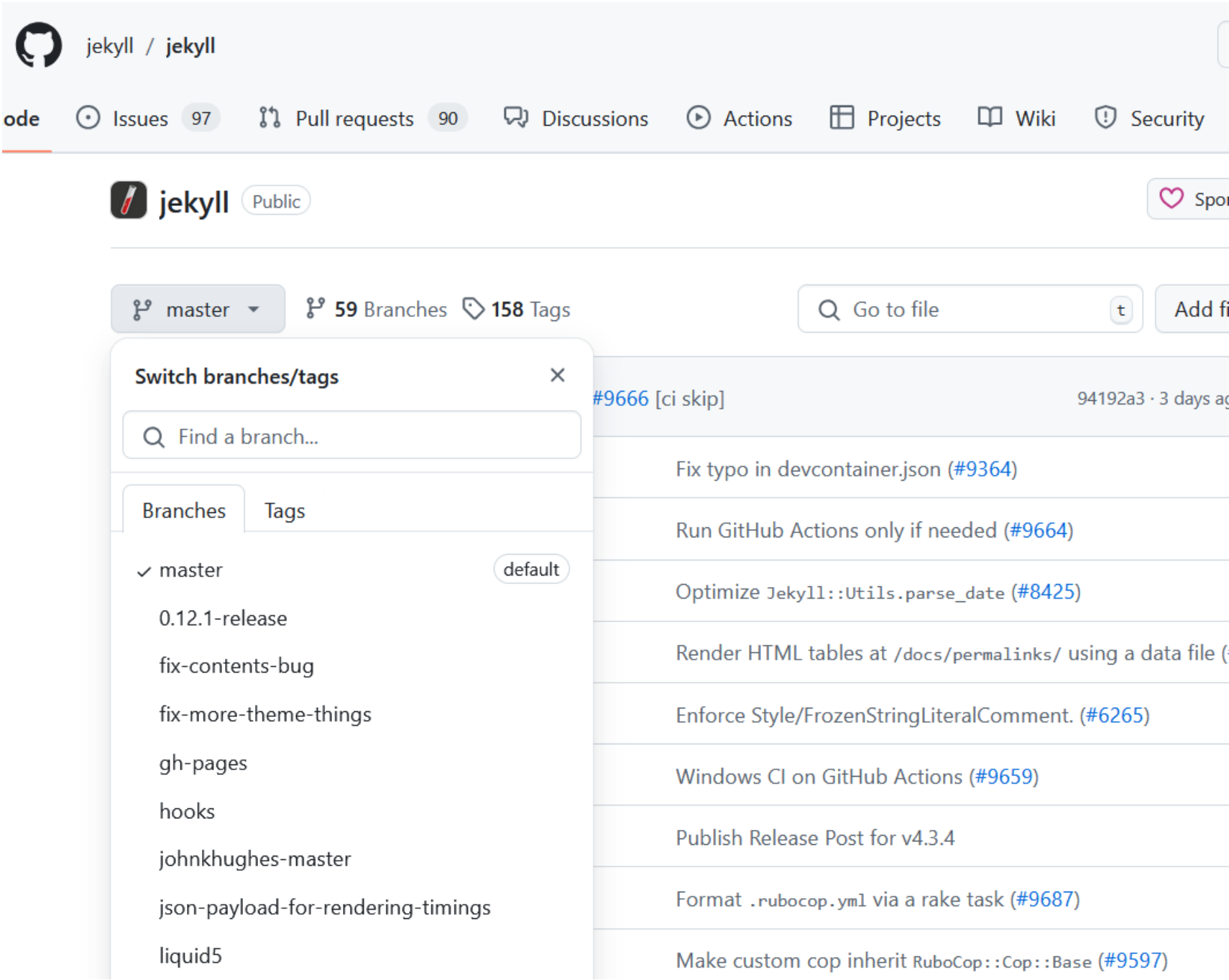
- 처음 작업한 저장소(main/master)를 여러 개 복사하여
각 작업자마다 다른 이름으로 버전관리하는 방법 (보통 feature 별로 관리)

Git Branch 기초



주요 브랜치 종류

- Master: 제품 출시용 브랜치
- Develop: 다음 버전 개발용 브랜치
- Feature: 기능 개발용 브랜치
- Release: 출시 준비용 브랜치
- Hotfix: 긴급 버그 수정용 브랜치



- ① 새로운 브랜치 feature_1 생성
- ② 생성한 브랜치 feature_1로 전환하기
- ③ 브랜치 사이의 커밋 차이 확인하기
 - feature_1, feature_2를 만들기
 - feature_2에는 없고 feature_1에만 있는 커밋 표시

Branch 명령어



Git 명령어

답

- ① 새로운 브랜치 feature_1 생성
- ② 생성한 브랜치 feature_1로 전환하기
- ③ 브랜치 사이의 커밋 차이 확인하기
 - feature_2에는 **없고** feature_1**에만 있는** 커밋 표시

git branch feature_1

git switch feature_1

git log feature_2..feature_1

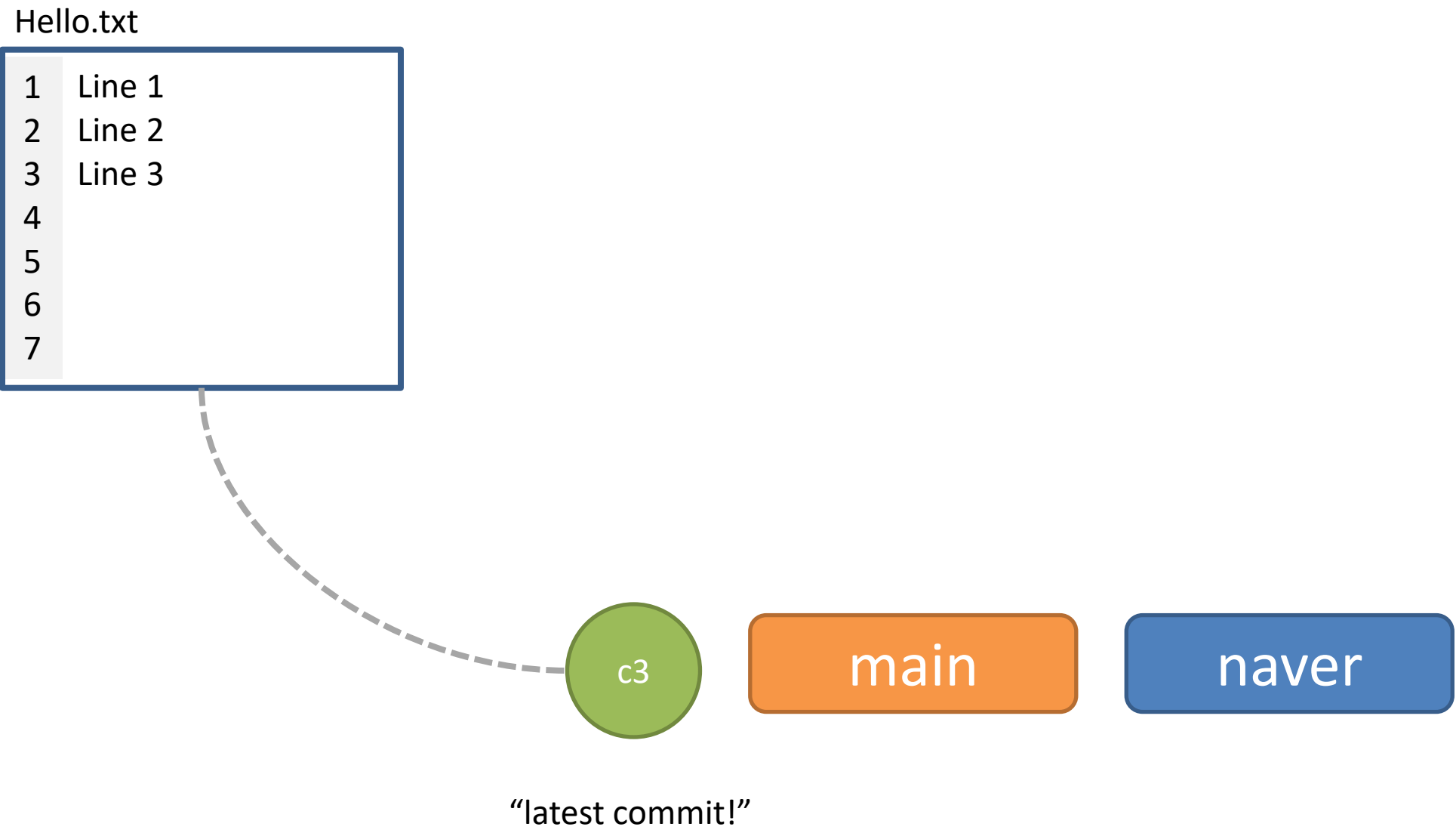
Branch 병합하기

한림대학교

Branch 병합하기



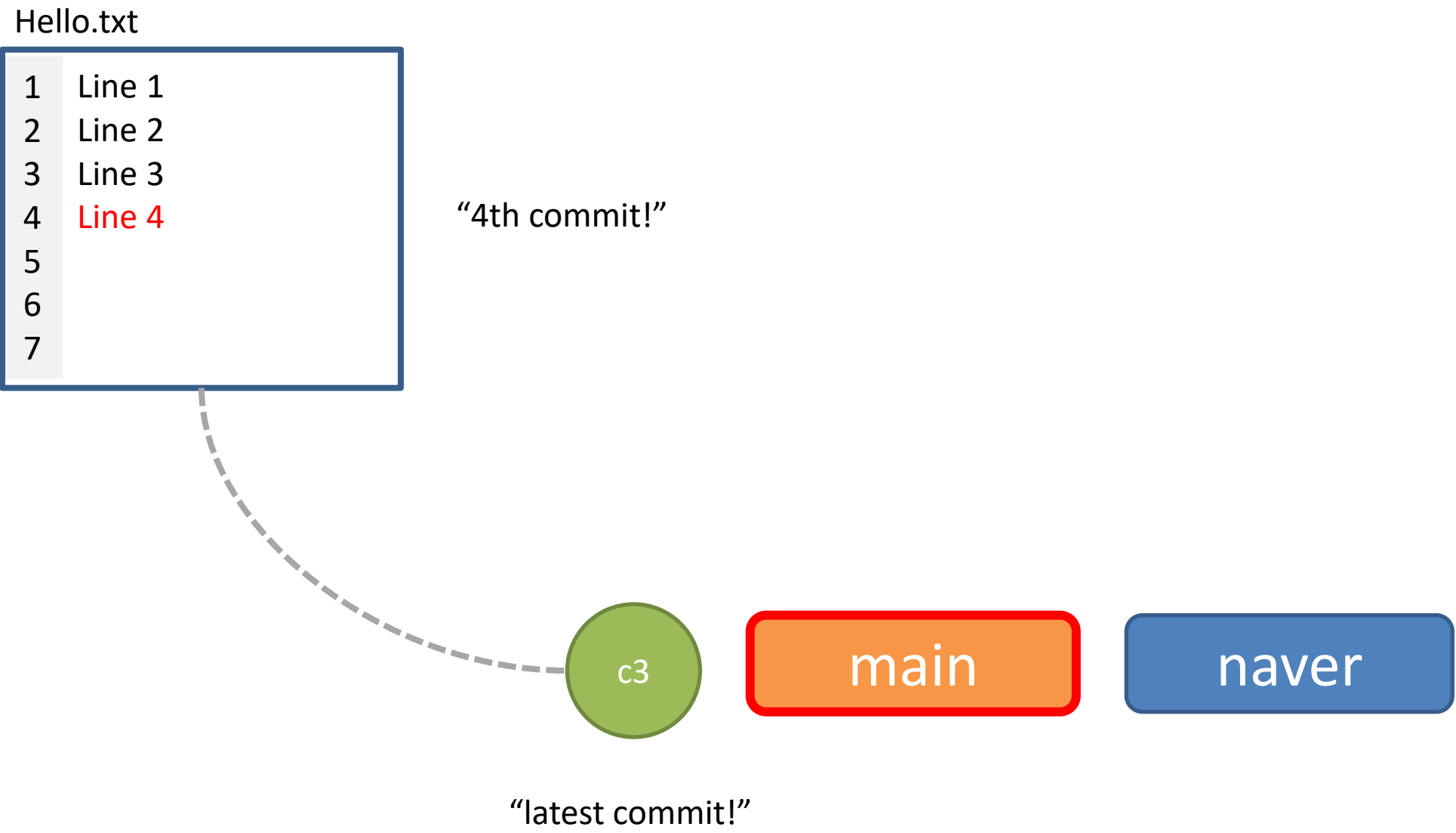
▶ 브랜치 병합하기



Branch 병합하기

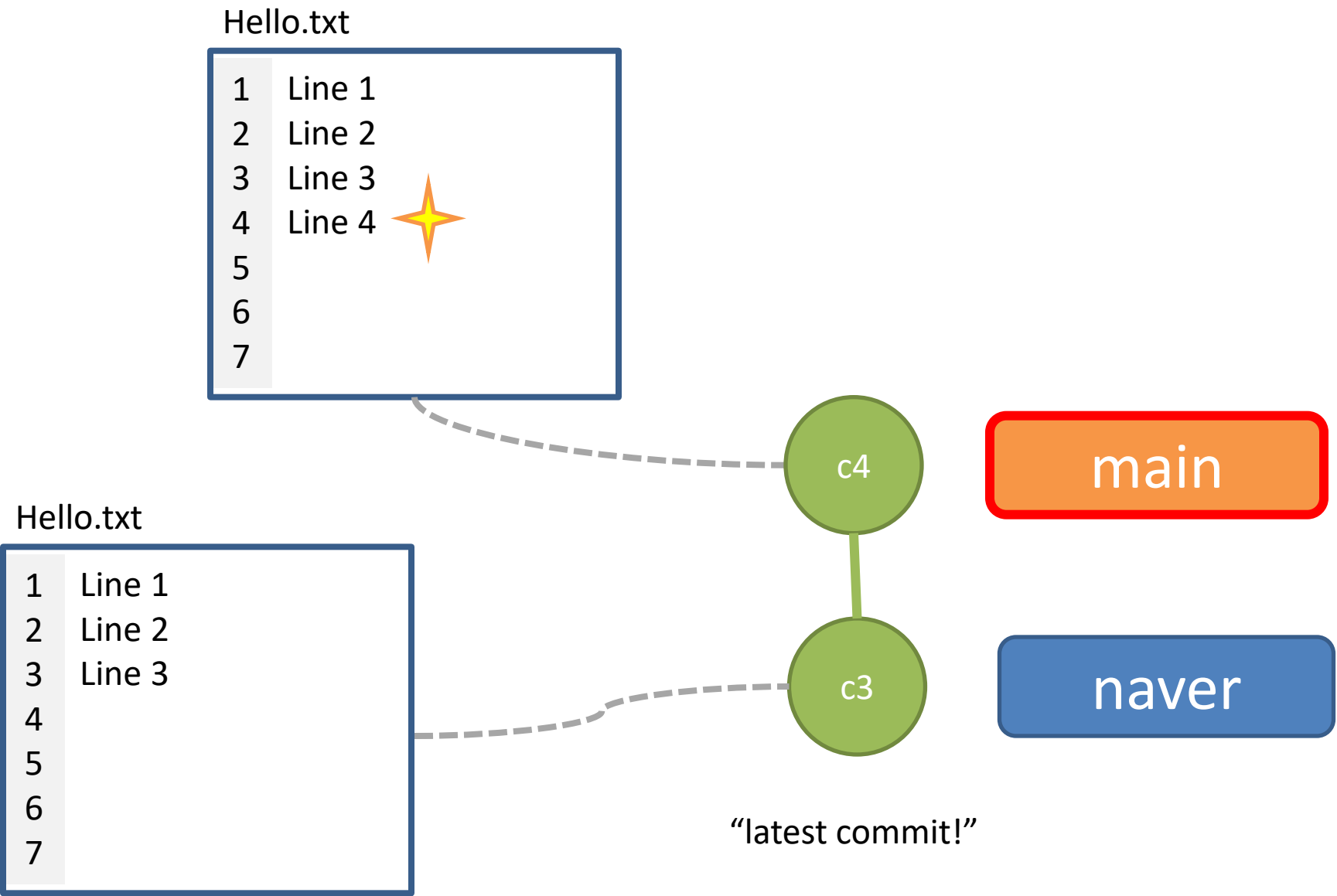


▶ 브랜치 병합하기



Branch 병합하기

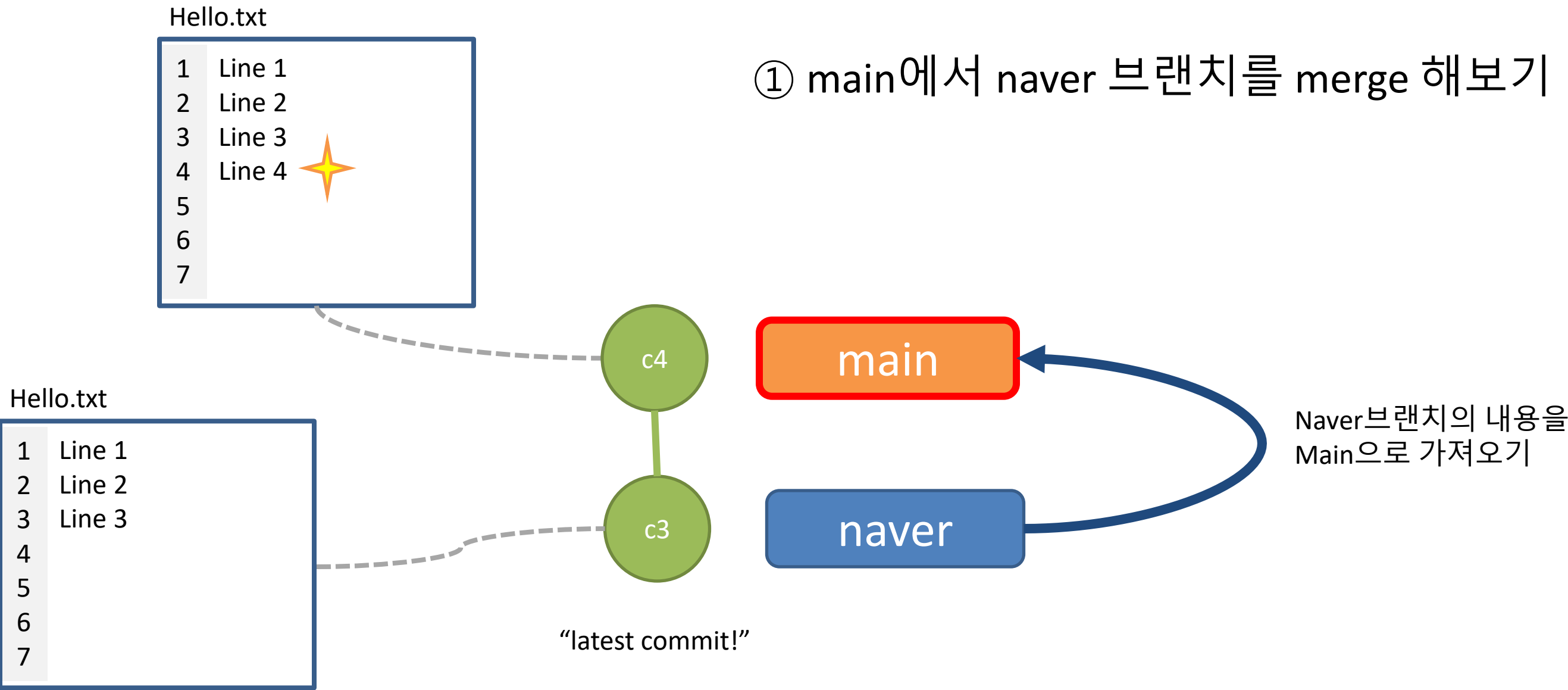
▶ 브랜치 병합하기



Branch 병합하기

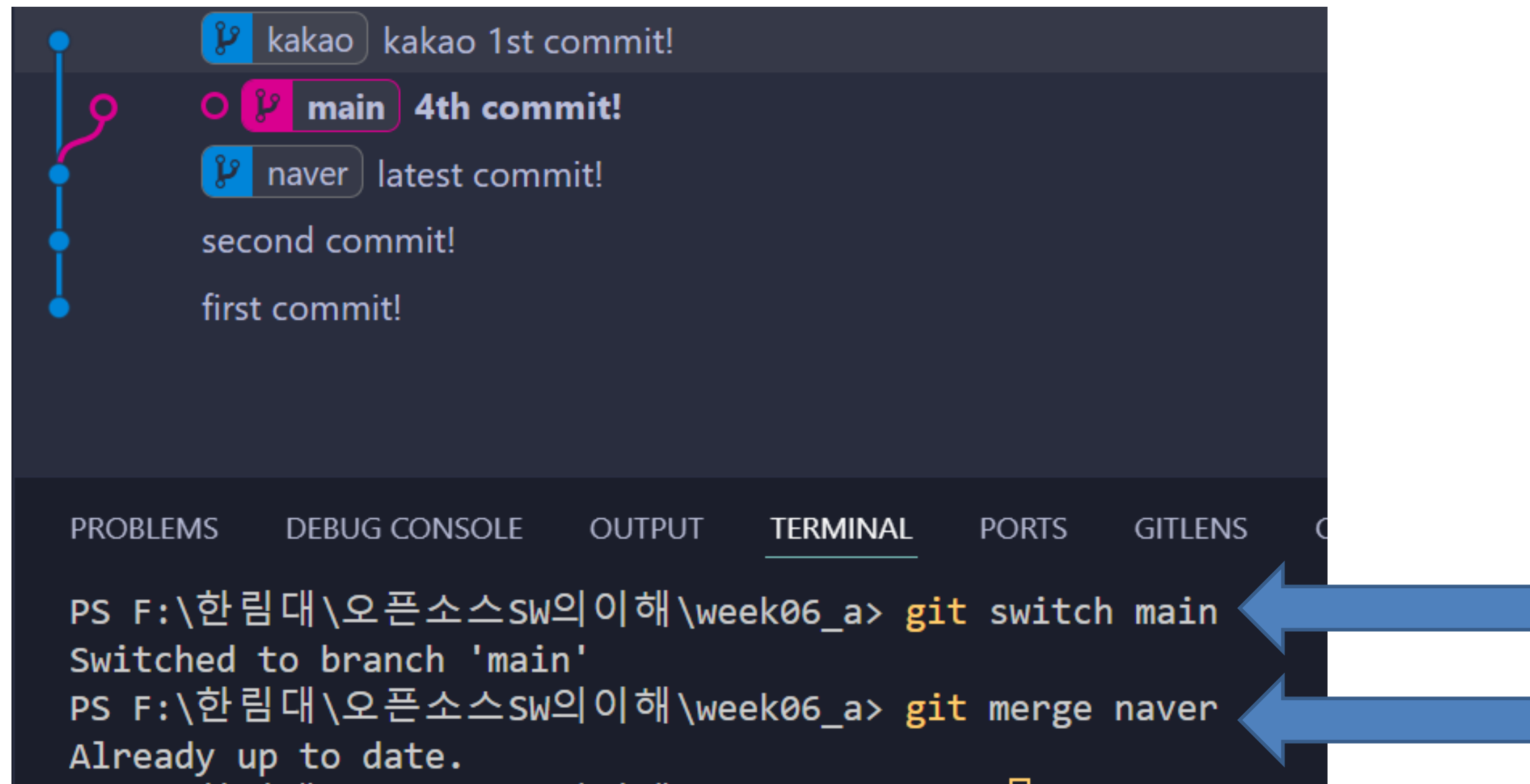


▶ 브랜치 병합하기



Branch 병합하기

▶ Git Branch Merge



The screenshot shows the VS Code Git interface. The left sidebar displays the commit history with a vertical line of blue dots. The main area shows the current branch 'main' at the '4th commit!'. Below this, the 'naver' branch is shown with its 'latest commit!'. The terminal window at the bottom shows the following commands and output:

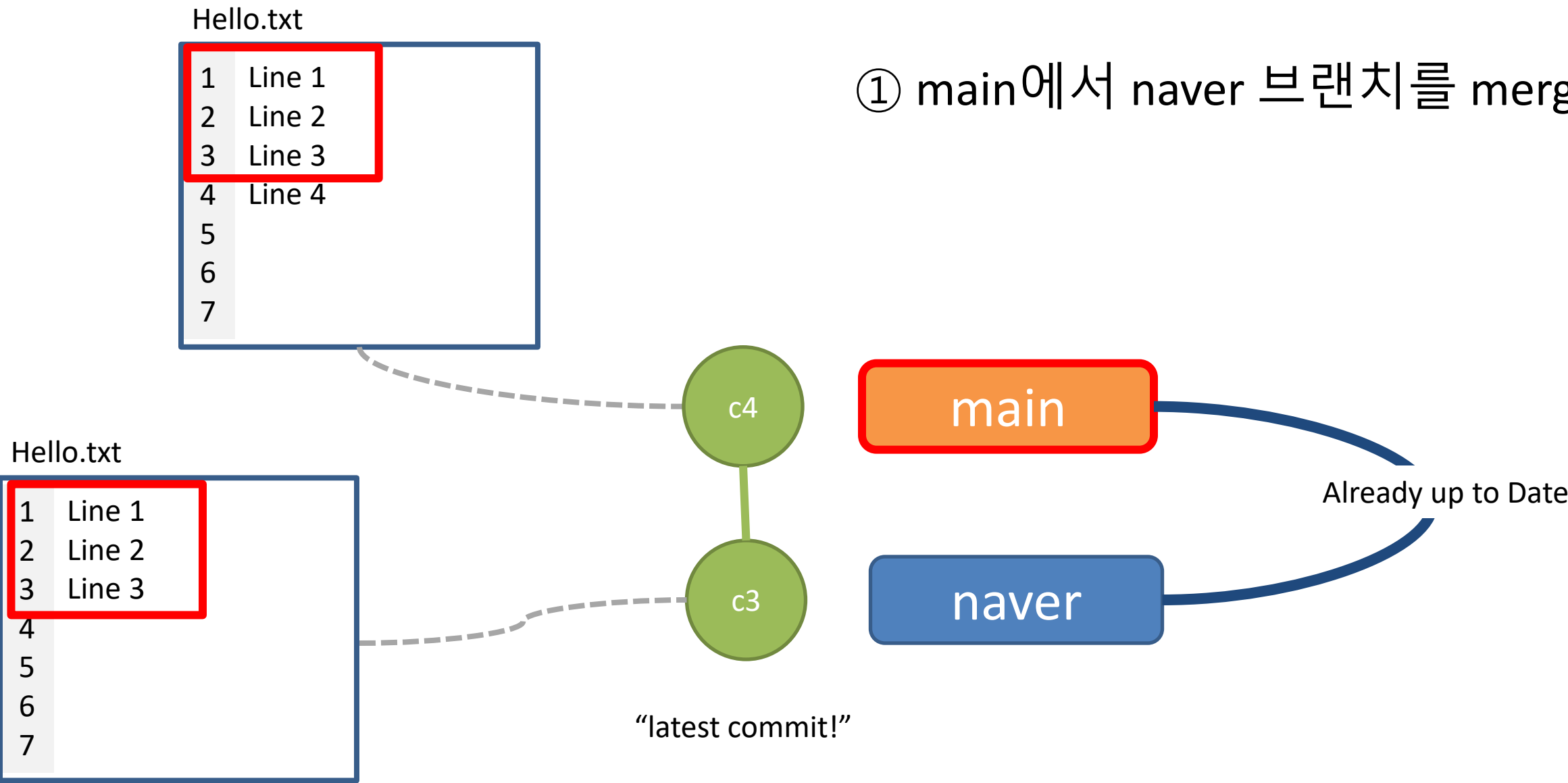
```
PS F:\한림대\오픈소스SW의 이해\week06_a> git switch main  
Switched to branch 'main'  
PS F:\한림대\오픈소스SW의 이해\week06_a> git merge naver  
Already up to date.
```

Two blue arrows point to the terminal output, highlighting the successful execution of the merge command.

Branch 병합하기



▶ 브랜치 병합하기

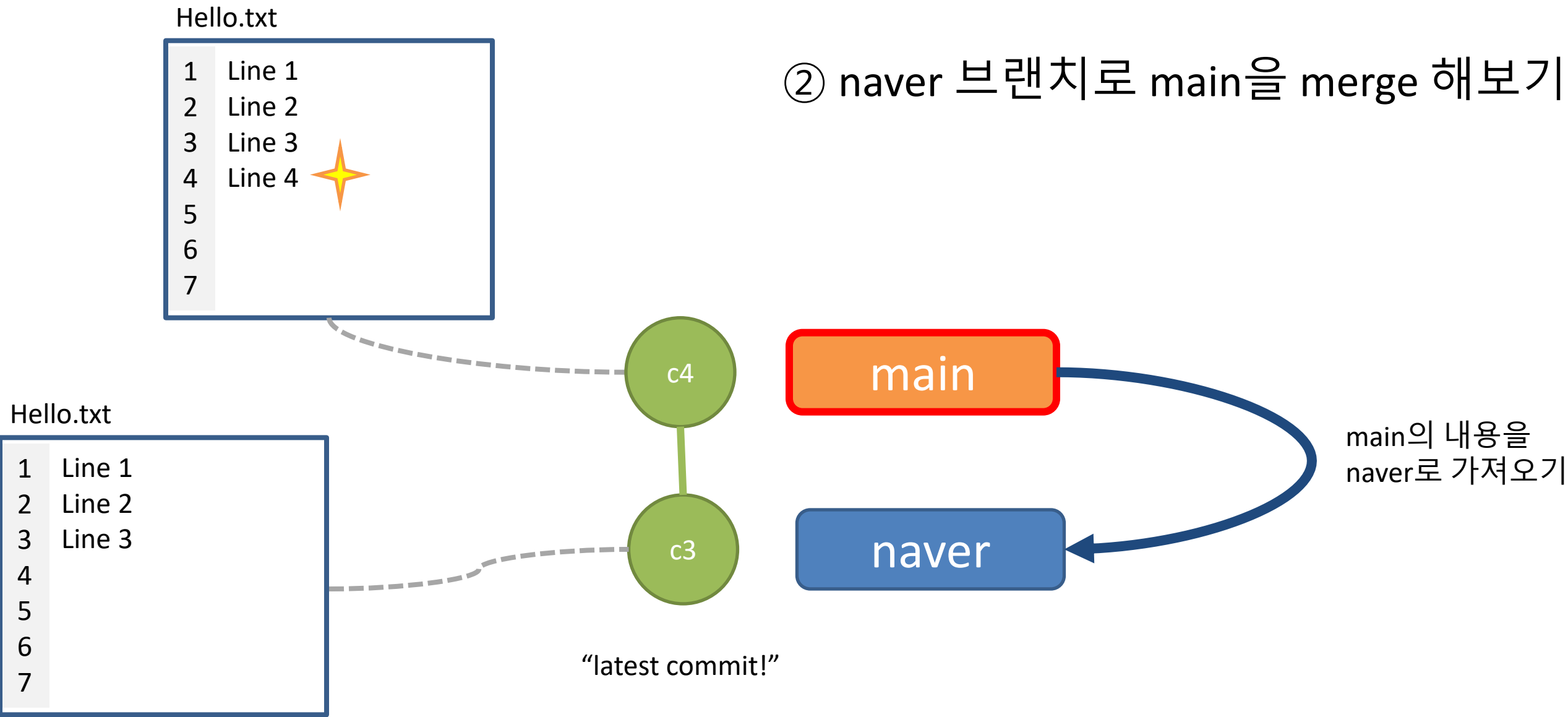


① main에서 naver 브랜치를 merge 해보기

Branch 병합하기



▶ 브랜치 병합하기



Branch 병합하기



Git Branch Merge

Graph



Description

 kakao kakao 1st commit!

 naver  main 4th commit!

latest commit!

second commit!

first commit!

PROBLEMS

DEBUG CONSOLE

OUTPUT

TERMINAL

PORTS

GITLENS

COMMENTS

```
PS F:\한림대\오픈소스SW의 이해\week06_a> git switch naver
Switched to branch 'naver'
PS F:\한림대\오픈소스SW의 이해\week06_a> git merge main
Updating 9ca778c..ae62cd2
Fast-forward
 test.txt | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

- ❖ Fast-Forward (빨리 감기) 병합
 - Branch를 분기한 이후 아무 변화가 없다면 (새로운 커밋x) Git에서 빠른 병합 가능
 - 포인터만 이동

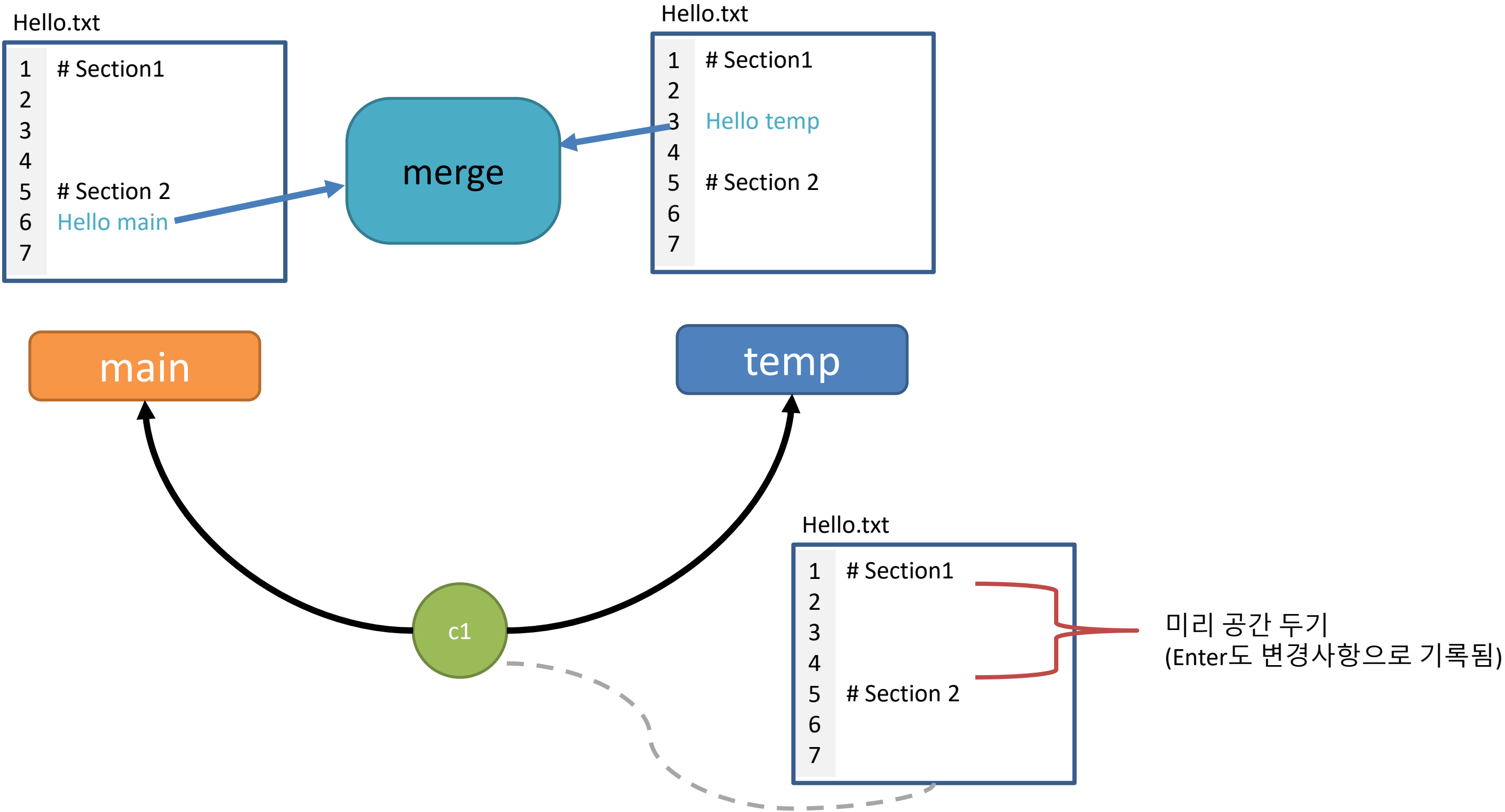
자동 병합하기 (Auto Merge)

한림대학교

자동 병합하기



▶ 다른 브랜치에서 다른 부분 수정한 후 병합

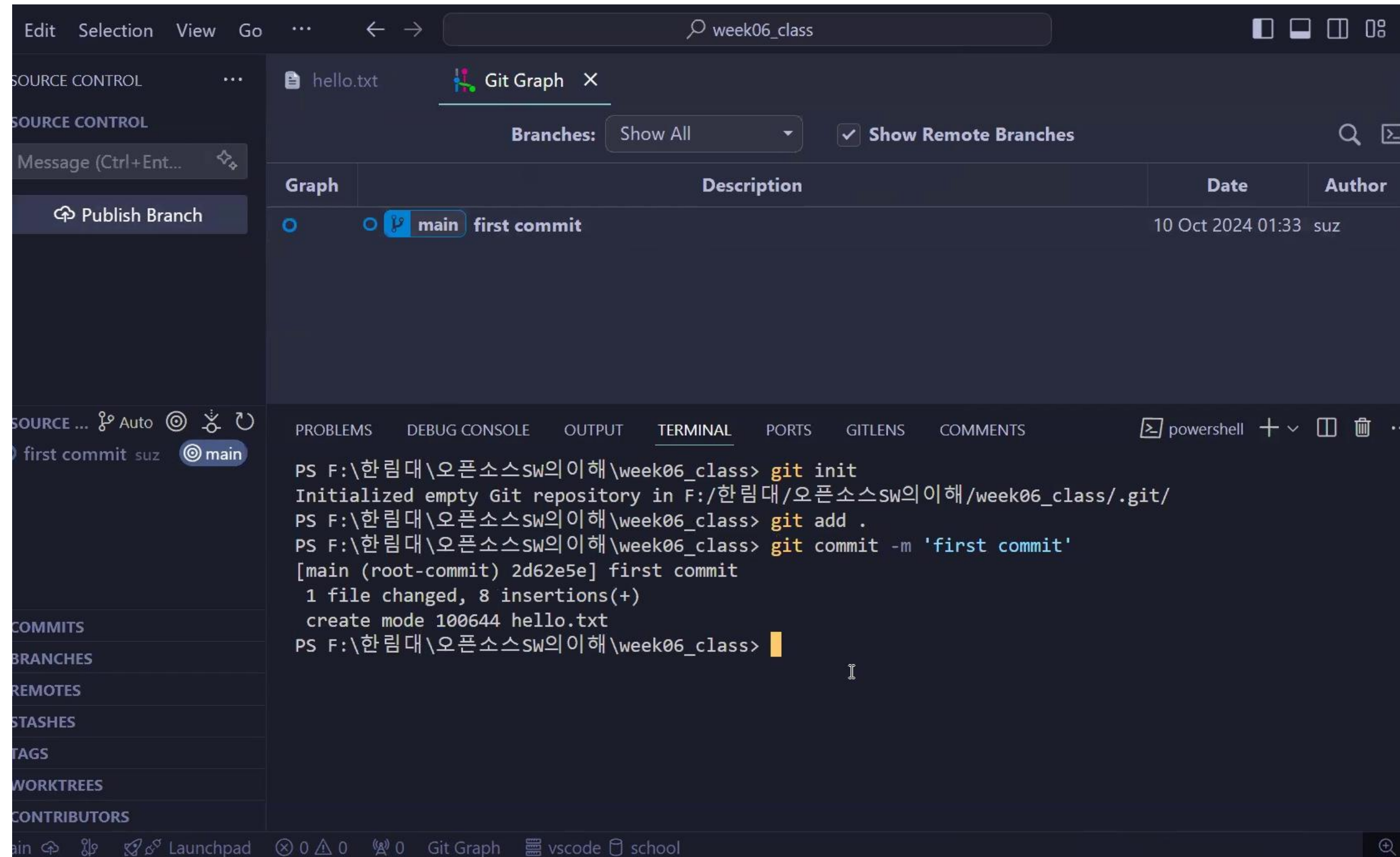


자동 병합하기

▶ 첫 번째 커밋 만들기

- ① 새로운 Git 저장소 생성
- ② Hello.txt 만들기
- ③ 파일 추가 + 커밋

- `git add .`
- `git commit -m "first commit"`



The screenshot displays the Visual Studio Code interface with the Git Graph extension open. The Git Graph view shows a single commit on the main branch, labeled 'first commit', dated '10 Oct 2024 01:33' by 'suz'. The Terminal window shows the following commands and output:

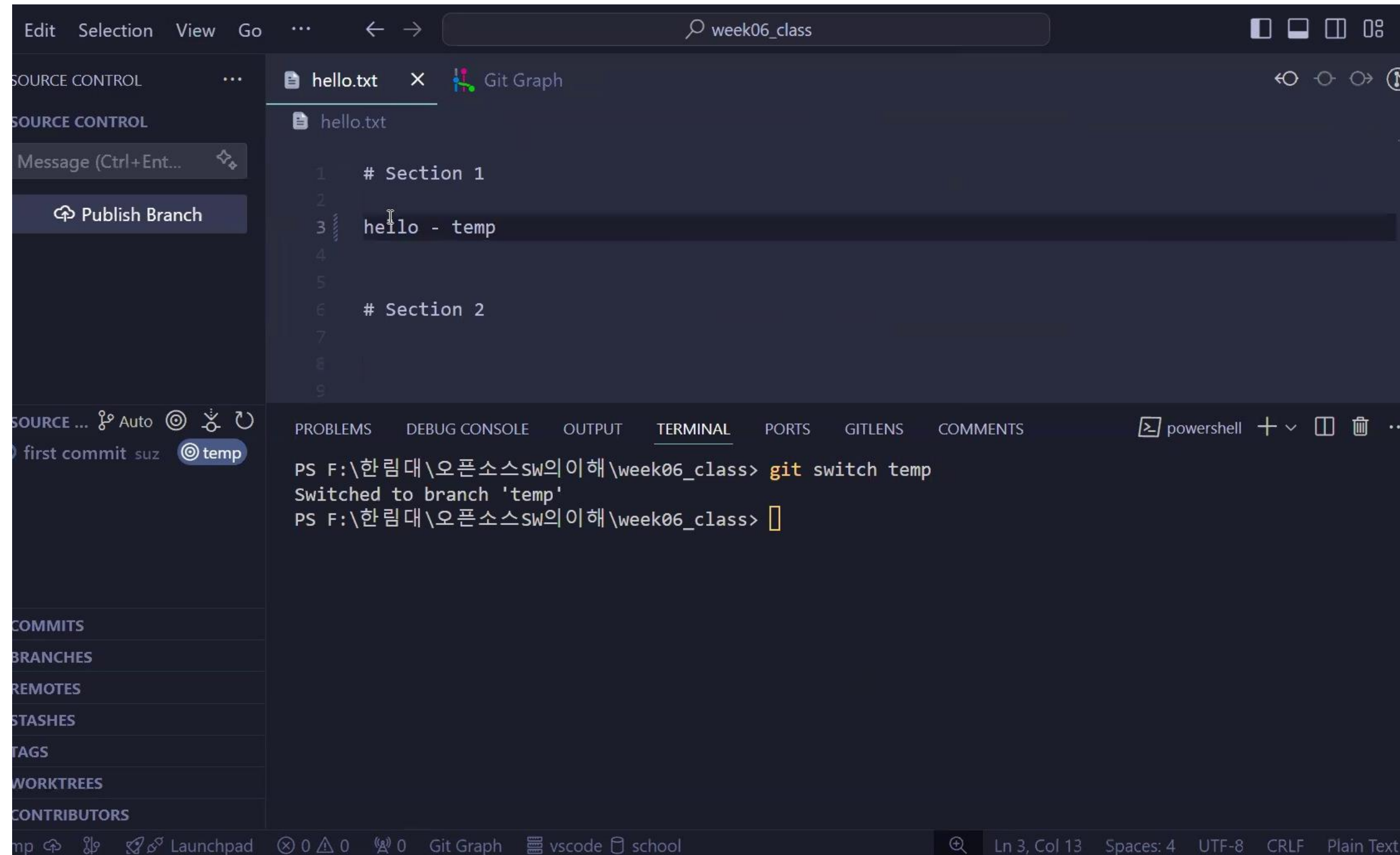
```
PS F:\한림대\오픈소스SW의이해\week06_class> git init
Initialized empty Git repository in F:/한림대/오픈소스SW의이해/week06_class/.git/
PS F:\한림대\오픈소스SW의이해\week06_class> git add .
PS F:\한림대\오픈소스SW의이해\week06_class> git commit -m 'first commit'
[main (root-commit) 2d62e5e] first commit
1 file changed, 8 insertions(+)
create mode 100644 hello.txt
PS F:\한림대\오픈소스SW의이해\week06_class>
```

자동 병합하기

▶ 새로운 브랜치 생성 및 전환

① 기능 분리: 새로운 브랜치 생성하고 전환

- git branch temp
- git switch temp



The screenshot shows the Visual Studio Code interface. The left sidebar displays the 'SOURCE CONTROL' view with a list of branches: 'first commit' and 'temp'. The 'temp' branch is currently selected. The main editor area shows a file named 'hello.txt' with the following content:

```
1 # Section 1
2
3 hello - temp
4
5
6 # Section 2
7
8
9
```

The bottom panel shows the 'TERMINAL' view with the following commands and output:

```
PS F:\한림대\오픈소스SW의이해\week06_class> git switch temp
Switched to branch 'temp'
PS F:\한림대\오픈소스SW의이해\week06_class>
```

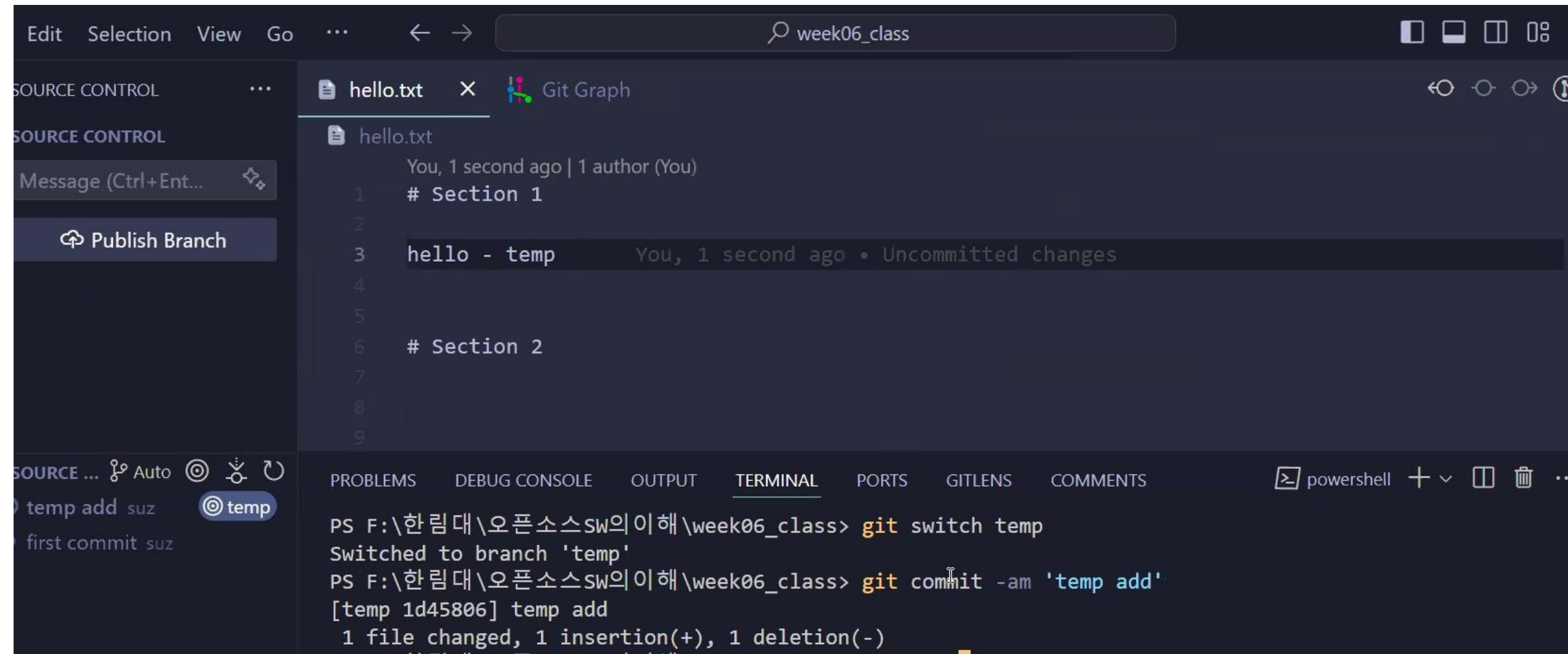
The status bar at the bottom indicates the current branch is 'temp' and the file is 'Ln 3, Col 13'.

자동 병합하기

▶ temp 브랜치에서 파일 수정 및 커밋

① `temp` 브랜치에서 파일을 수정
+ 변경사항 커밋

- `git commit -am 'temp add'`



The screenshot shows the Visual Studio Code interface. The editor window displays the file `hello.txt` with the following content:

```
1 # Section 1
2
3 hello - temp
4
5
6 # Section 2
7
8
9
```

The status bar at the bottom indicates "You, 1 second ago | 1 author (You)" and "Uncommitted changes". The terminal window at the bottom shows the following commands and output:

```
PS F:\한림대\오픈소스SW의이해\week06_class> git switch temp
Switched to branch 'temp'
PS F:\한림대\오픈소스SW의이해\week06_class> git commit -am 'temp add'
[temp 1d45806] temp add
1 file changed, 1 insertion(+), 1 deletion(-)
```

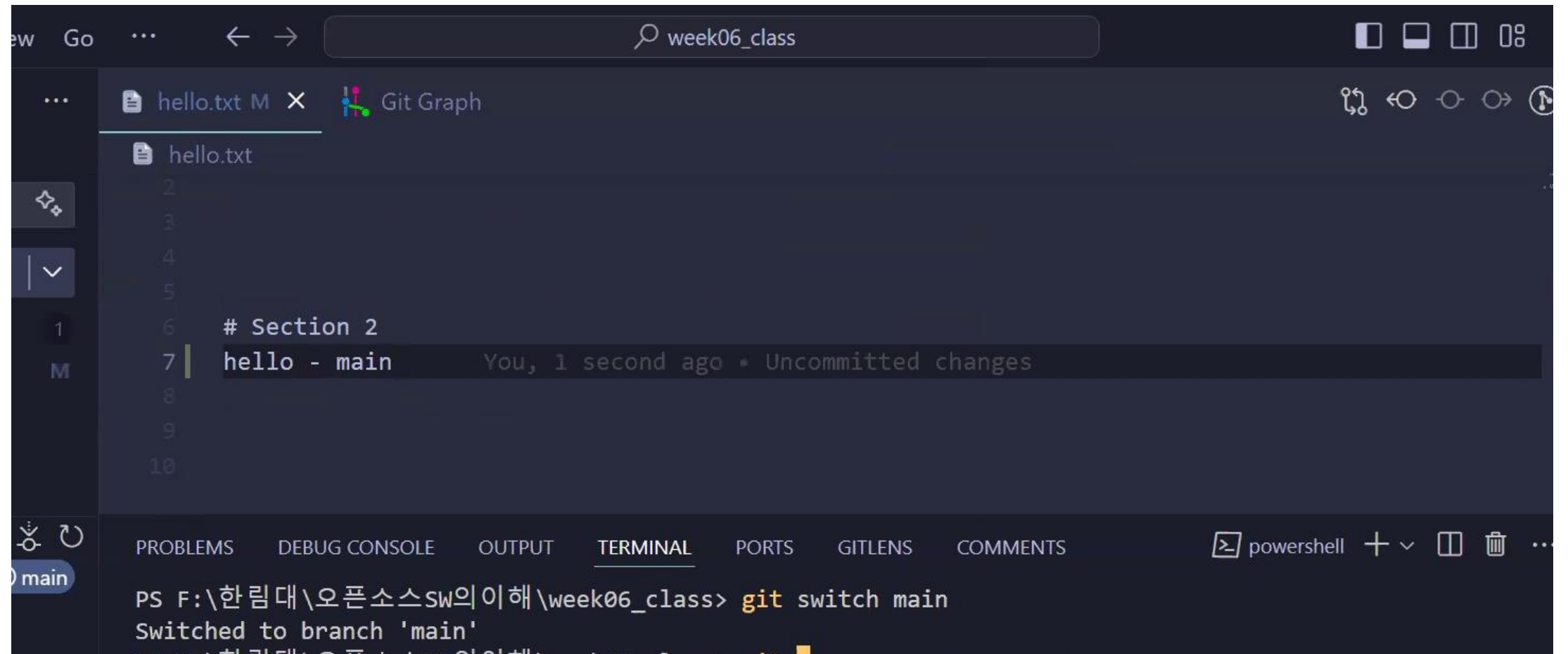
자동 병합하기

▶ main 브랜치로 복귀

① `main` 브랜치로 복귀!!!

② `main` 브랜치에서도
파일 수정 + 커밋

- git switch main
- git commit -am 'main hello'



The screenshot shows the Visual Studio Code interface. The file explorer on the left shows a file named 'hello.txt'. The editor window displays the content of 'hello.txt', which includes a comment '# Section 2' and a line 'hello - main'. A status bar at the bottom of the editor indicates 'You, 1 second ago • Uncommitted changes'. The terminal at the bottom shows the command 'git switch main' being executed, with the output 'Switched to branch 'main''.

```
PS F:\한림대\오픈소스SW의이해\week06_class> git switch main
Switched to branch 'main'
```

자동 병합하기

▶ 브랜치 상태 확인

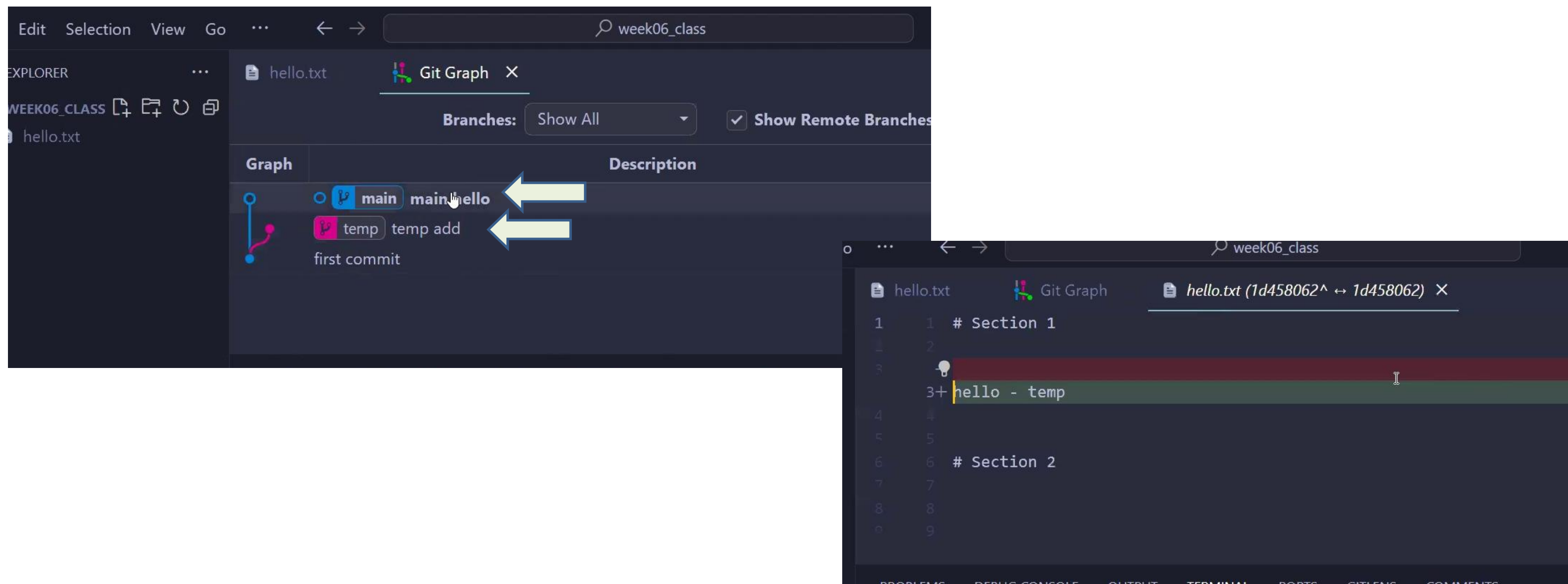
The screenshot displays the VS Code interface with the Git Graph extension. The 'Git Graph' panel is open, showing a graph of commits and branches. A red box highlights the 'main' branch, which is currently checked out. The 'temp' branch is also visible, with a commit 'temp add' shown. The 'first commit' is the root of the 'main' branch. The terminal at the bottom shows the following commands and output:

```
PS F:\한림대\오픈소스SW의이해\week06_class> git switch main
Switched to branch 'main'
PS F:\한림대\오픈소스SW의이해\week06_class> git commit -am 'main hello'
[main ed0f39f] main hello
1 file changed, 1 insertion(+)
PS F:\한림대\오픈소스SW의이해\week06_class>
```

자동 병합하기

▶ 병합 준비

❖브랜치 변경사항 확인



자동 병합하기

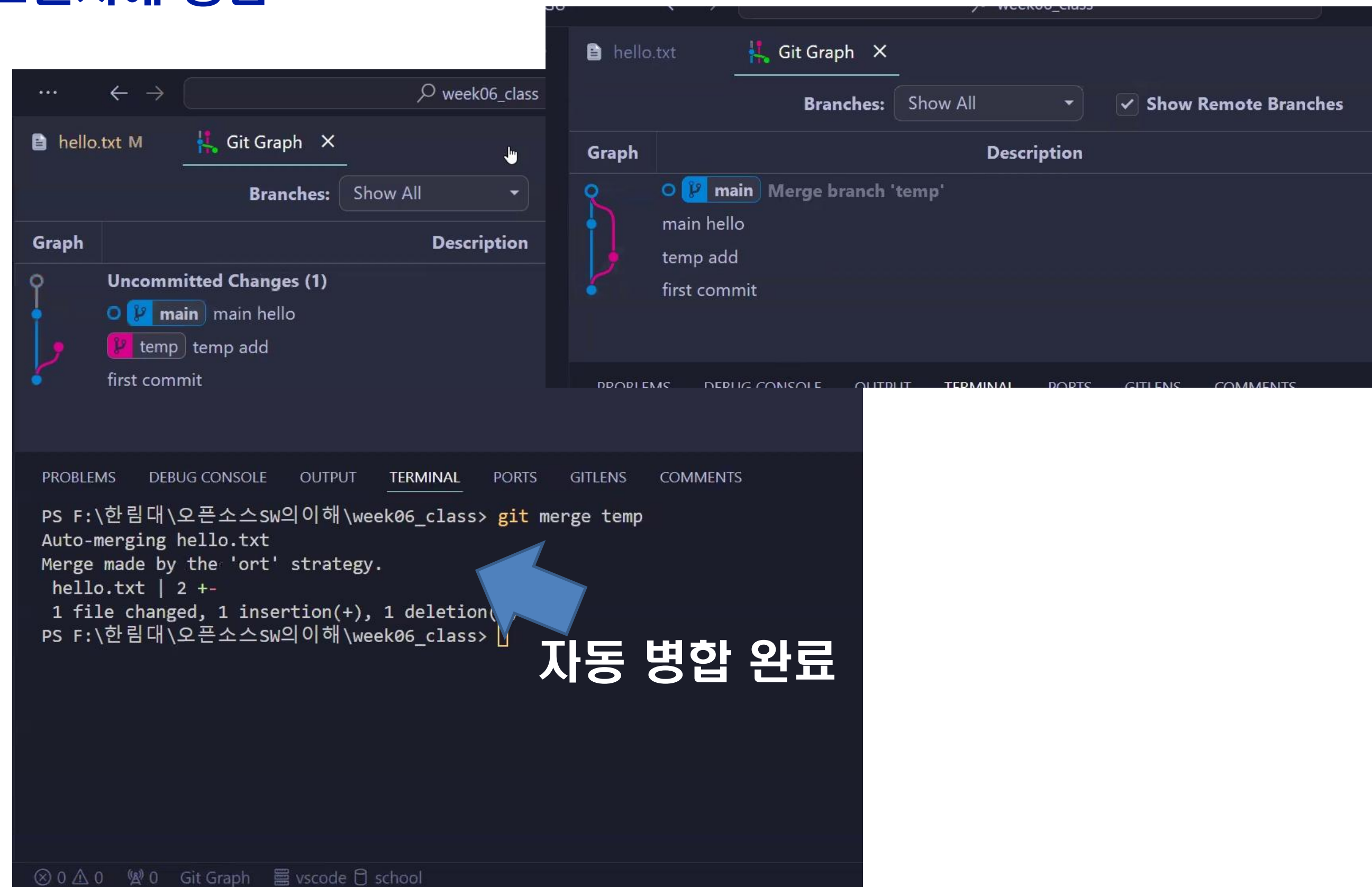
▶ temp 브랜치를 main 브랜치에 병합

① `main` 브랜치에서
`temp` **브랜치 병합**

▪ git merge temp

※ 브랜치 삭제

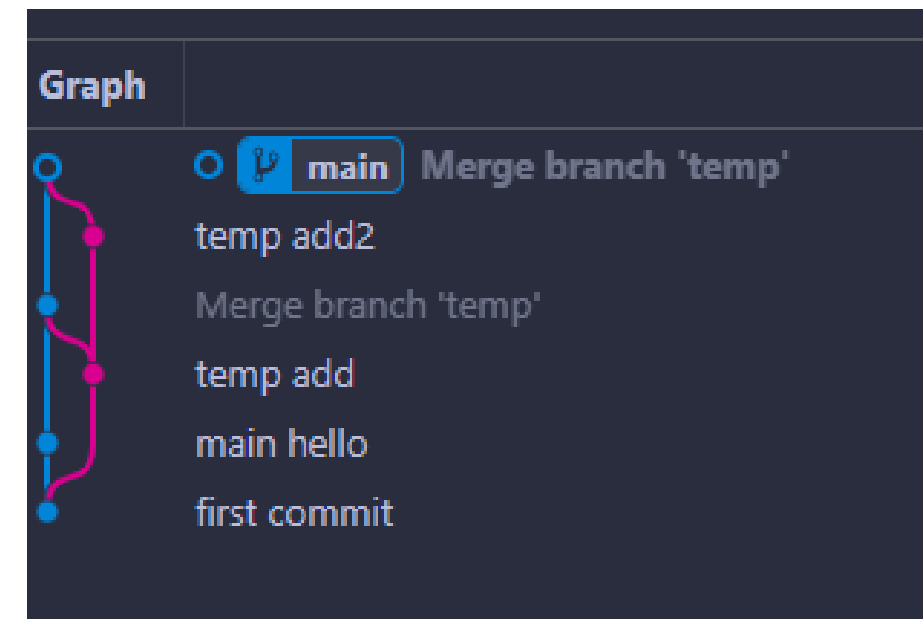
▪ git branch -D [브랜치명]



CLASSUM

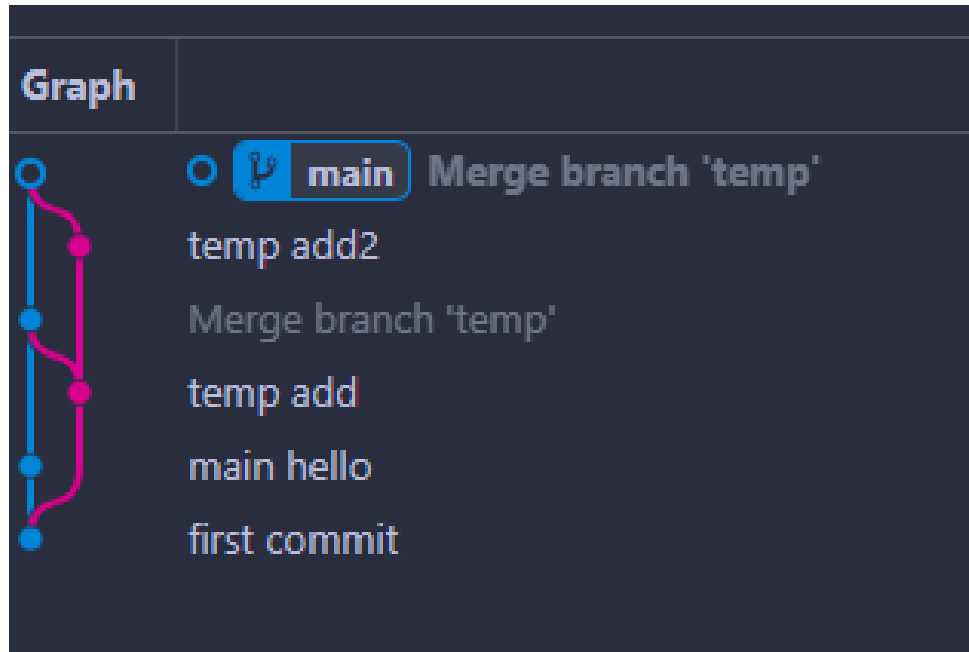
- ① temp branch에서 새로운 txt파일 만들고 커밋하기
- ② main으로 merge하기
- ③ ``git branch -D [브랜치명]`` 으로 temp 브랜치 삭제

➤ Git Graph 스크린샷 올리기



자동 병합하기

CLASSSUM



답

```
PS F:\한림대\오픈소스SW의이해\week06_b> git switch temp  
Switched to branch 'temp'
```

```
PS F:\한림대\오픈소스SW의이해\week06_b> git add .  
PS F:\한림대\오픈소스SW의이해\week06_b> git commit -am 'temp add2'  
[temp 6f9025c] temp add2  
1 file changed, 1 insertion(+)  
create mode 100644 t.txt
```

```
PS F:\한림대\오픈소스SW의이해\week06_b> git switch main  
Switched to branch 'main'  
PS F:\한림대\오픈소스SW의이해\week06_b> git merge temp  
Merge made by the 'ort' strategy.  
t.txt | 1 +  
1 file changed, 1 insertion(+)  
create mode 100644 t.txt
```

```
PS F:\한림대\오픈소스SW의이해\week06_b> git branch -D temp  
Deleted branch temp (was 6f9025c).
```

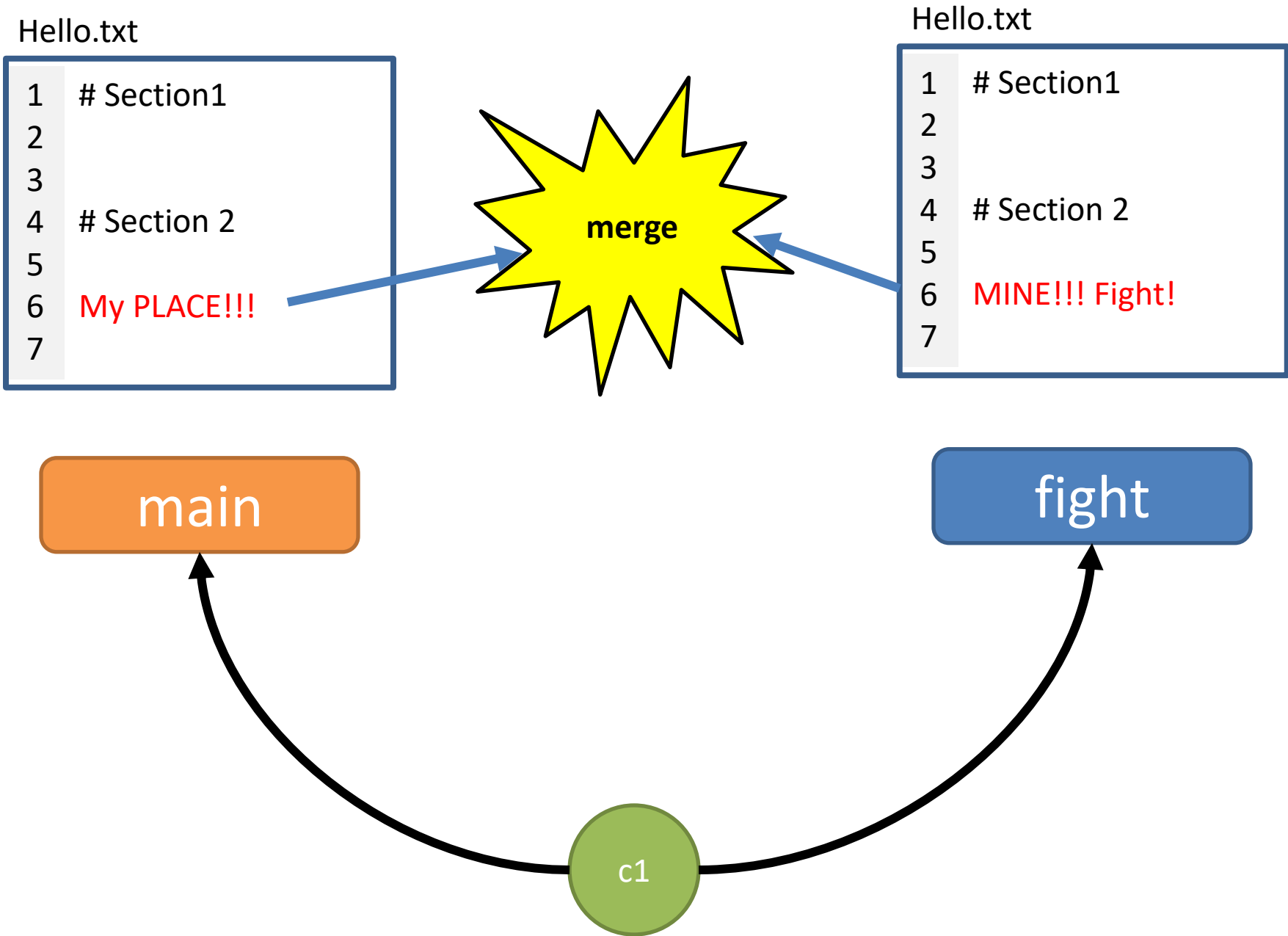
충돌 해결하기

한림대학교

충돌 해결하기

I

▶ 다른 브랜치에서 같은 부분 수정한 후 병합

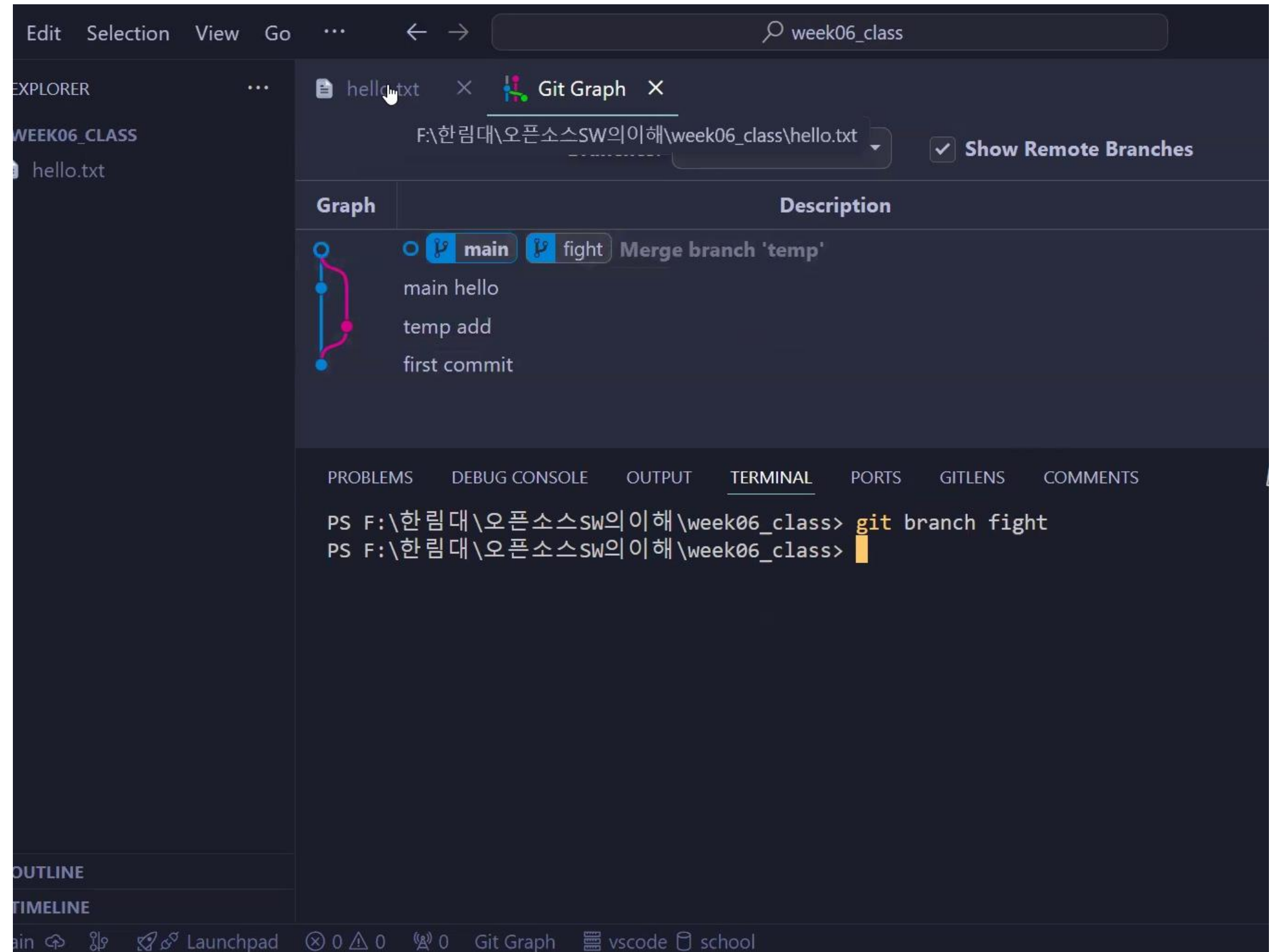


충돌 해결하기

▶ 새로운 브랜치 생성하기

① 새로운 브랜치 `fight`를 생성

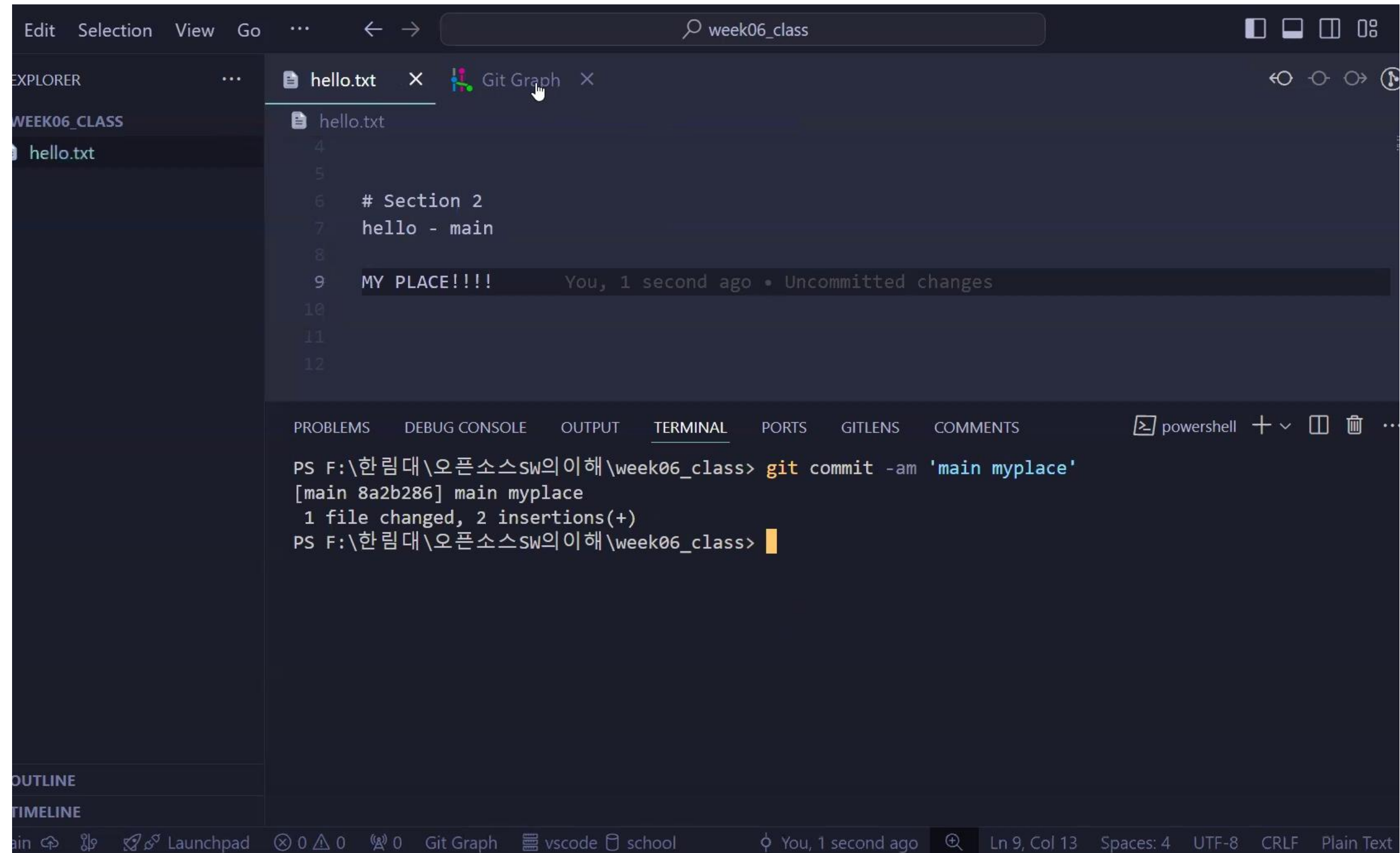
- git branch fight



충돌 해결하기

▶ 변경 사항 커밋하기

- ① main에서 글 추가!
- ② 커밋



The screenshot shows the Visual Studio Code interface with a file named `hello.txt` open. The file content is as follows:

```
4  
5  
6 # Section 2  
7 hello - main  
8  
9 MY PLACE!!!!  
10  
11  
12
```

A status bar at the bottom of the editor indicates "You, 1 second ago • Uncommitted changes". The terminal at the bottom shows the following commands and output:

```
PS F:\한림대\오픈소스SW의이해\week06_class> git commit -am 'main myplace'  
[main 8a2b286] main myplace  
1 file changed, 2 insertions(+)  
PS F:\한림대\오픈소스SW의이해\week06_class>
```

The status bar at the very bottom of the window shows "You, 1 second ago" and "Ln 9, Col 13 Spaces: 4 UTF-8 CRLF Plain Text".

충돌 해결하기

▶ 변경 사항 커밋하기

- ① Fight 브랜치에서 작업
- ② 파일 내용 추가

- git switch fight
- git commit -am 'fights place'

```
Graph      Description
main myplace
fight Merge branch 'temp'
main hello
temp add
first commit

PROBLEMS  DEBUG CONSOLE  OUTPUT  TERMINAL  PORTS  GITLENS  COMM
PS F:\한림대\오픈소스SW의이해\week06_class> git switch fight
Switched to branch 'fight'
PS F:\한림대\오픈소스SW의이해\week06_class>

hello.txt M x  Git Graph  hello.txt (8a2b286d^ ↔ 8a2b286d)
hello.txt
4
5
6 # Section 2
7 hello - main
8
9 MINE!!! - fight  You, 1 second ago • Uncommitted changes
10

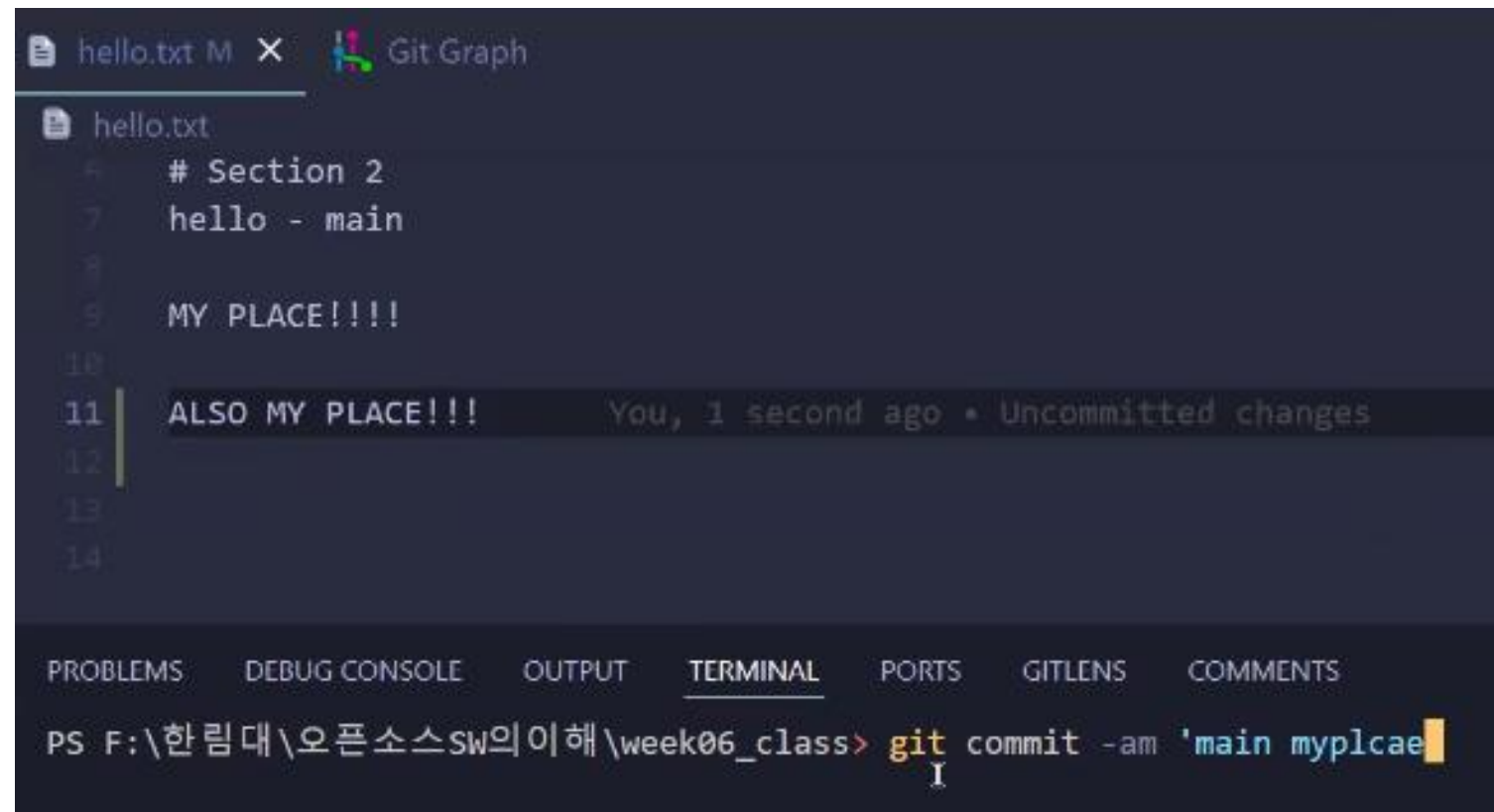
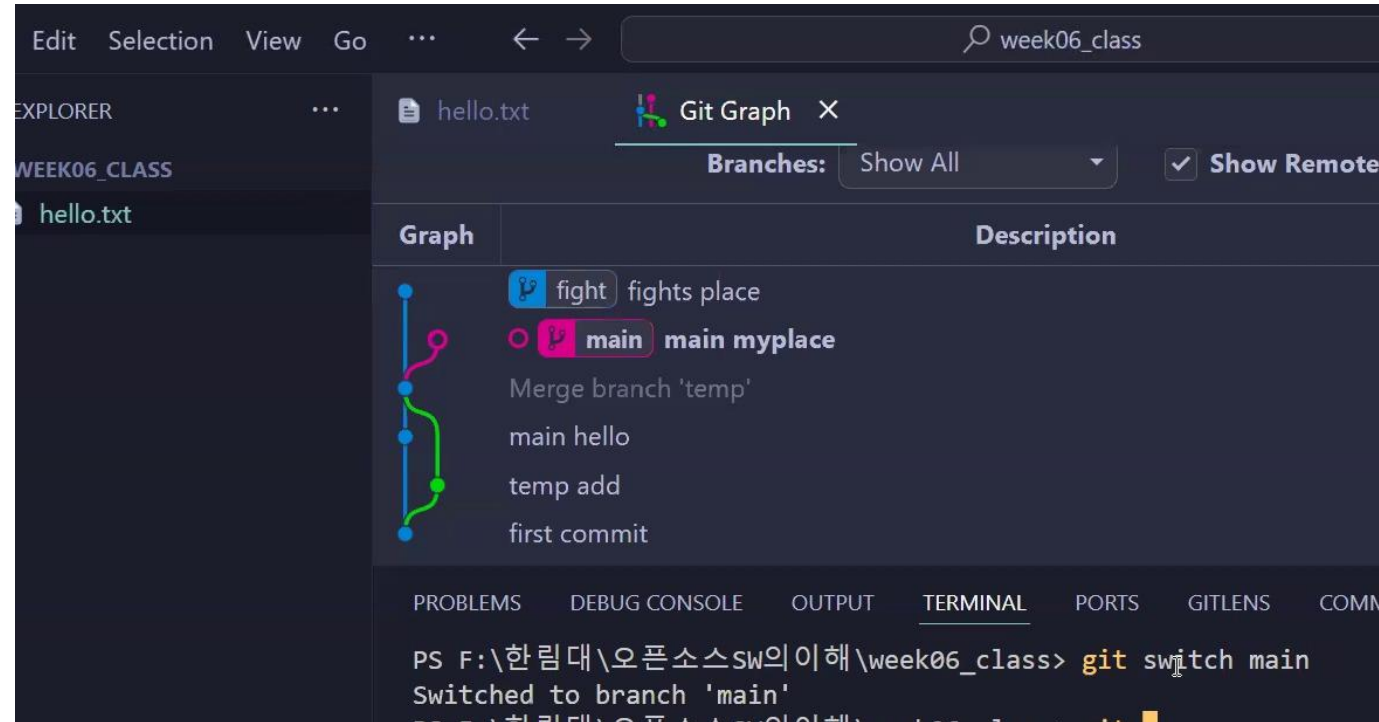
PROBLEMS  DEBUG CONSOLE  OUTPUT  TERMINAL  PORTS  GITLENS  COMMENTS
PS F:\한림대\오픈소스SW의이해\week06_class> git switch fight
Switched to branch 'fight'
PS F:\한림대\오픈소스SW의이해\week06_class> git commit -am 'fi
```

충돌 해결하기



▶ 메인 브랜치로 돌아가서 한 번 더 커밋하기

- `git switch main`
- `git commit -am 'main myplace2'`



충돌 해결하기



▶ 병합을 위한 변경 사항 커밋하기

EXPLORER

WEEK06_CLASS

hello.txt

hello.txt

Git Graph

Branches: Show All

Show Remote Branches

Graph	Description
	main myplace
	fight fights place
	main myplace
	Merge branch 'temp'
	main hello
	temp add

PROBLEMS

DEBUG CONSOLE

OUTPUT

TERMINAL

PORTS

GITLENS

COMMENTS

PS F:\한림대\오픈소스SW의 이해\week06_class> git commit -am 'main myplace'

[main d65f9cc] main myplace

1 file changed, 2 insertions(+)

PS F:\한림대\오픈소스SW의 이해\week06_class>

충돌 해결하기

▶ 브랜치 병합하기

① `main` 브랜치에서
`fight` 브랜치를
병합 시도

▪ git merge fight

❖ 충돌 발생 확인 메시지:

- Automatic merge failed;
fix conflicts and then
commit the result.

```

6  # Section 2
7  hello - main
8
9  <<<<<< HEAD (Current Change)
10 MY PLACE!!!!
11      You, 3 minutes ago • main myplace
12 >>>>>> fight (Incoming Change)
13
14
15 =====
16 MINE!!! - fight
17 >>>>>> fight (Incoming Change)
18

```

Resolve in Merge Editor

PROBLEMS DEBUG CONSOLE OUTPUT TERMINAL PORTS GITLENS COMMENTS

```

Auto-merging hello.txt
CONFLICT (content): Merge conflict in hello.txt
Automatic merge failed; fix conflicts and then commit the result.
PS F:\한림대\오픈소스SW의이해\week06_class>

```

충돌 해결하기

충돌 확인 및 해결

❖ 충돌이 발생한 파일 'hello.txt'를 열면 다음과 같은 표시를 확인 가능:

```
<<<<<<< HEAD
현재 변경 내용 (main 브랜치)
=====
들어오는 변경 내용 (fight 브랜치)
>>>>>>> fight
```

- VS Code는 충돌 부분에 대해
 - 1) Accept Current Change(현재 변경 사항 수락): : `main` 브랜치의 변경 내용 유지
 - 2) Accept Incoming Change (들어오는 변경 사항 수락): `fight` 브랜치의 변경 내용 적용
 - 3) Accept Both Changes,
 - 4) Compare Changes
 - 5) Ignore (Resolve Manager) 등의 옵션 제공

```
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
<<<<<<< HEAD (Current Change)
MY PLACE!!!!

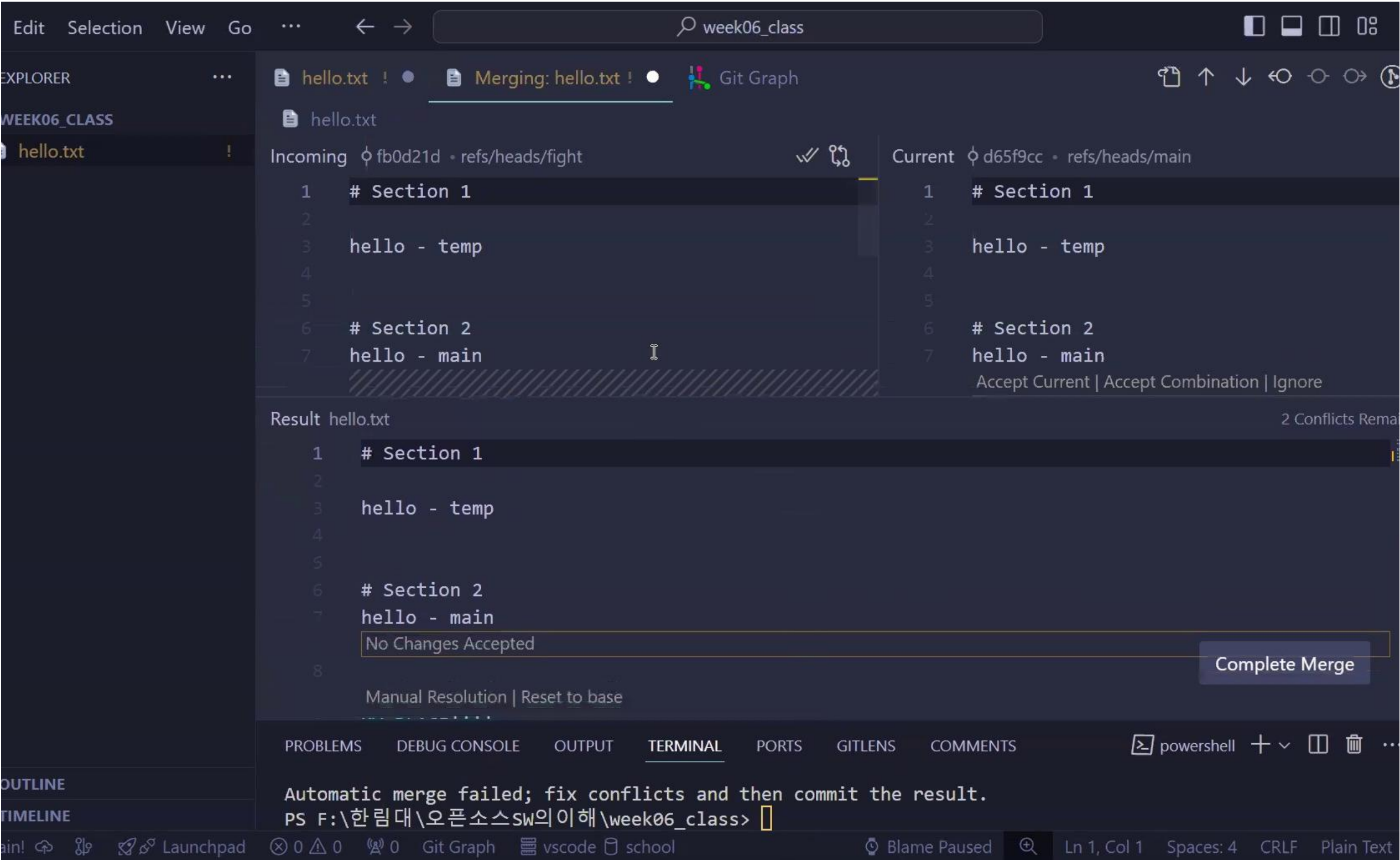
ALSO MY PLACE!!!      You, 1 minute ago • main myplace

=====
MINE!!! - fight
>>>>>>> fight (Incoming Change)
```

충돌 해결하기

충돌 해결하기

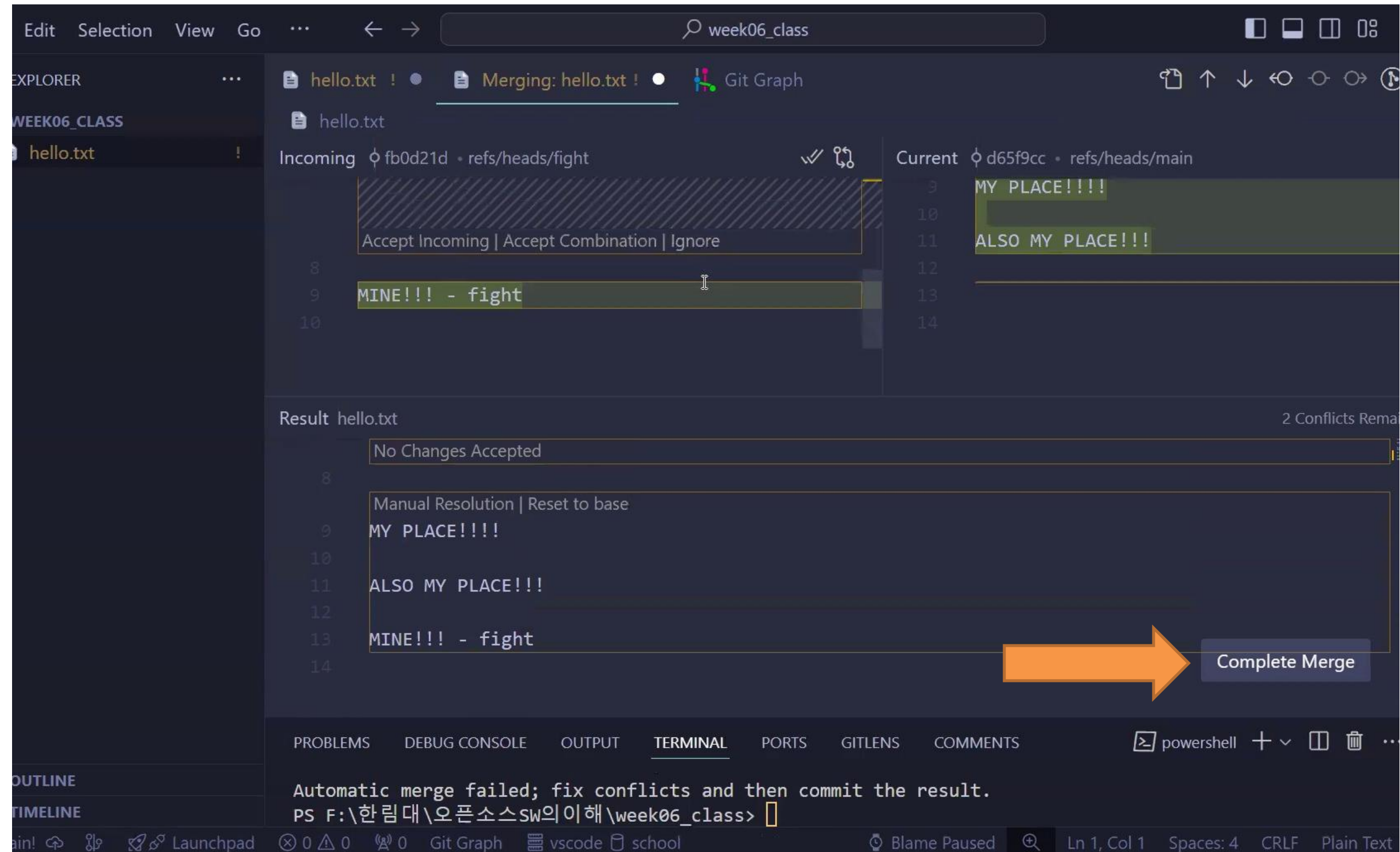
Resolve Manager ➡



충돌 해결하기

▶ 병합 완료 및 커밋하기

- ✓ 충돌을 모두 해결한 후
- ✓ 변경된 파일을 추가하고 커밋



브랜치 병합 실습

CLASSUM

Festival Booth 앱 출시 미션

학교 축제 방문자가 부스를 찾고
방문 코스를 저장할 수 있는
Streamlit 앱을 완성

공통 개발 환경

Python, Streamlit, pandas

시작 파일

app.py
booths.csv
requirements.txt
README.md

출시 성공 조건

- ✓ 실내 부스 검색
- ✓ 방문 코스 선택
- ✓ CSV 저장

브랜치 병합 실습

브랜치별 기능 개발

팀원들과 상의해서 기능 분담

각자 feature/<기능>_<이니셜> 브랜치에서 기능 **하나**를 구현하고 커밋

- 1) 검색·필터 분야와 실내 여부로 부스 목록 좁히기
- 2) 랜덤 추천 조건에 맞는 부스 한 곳 보여주기
- 3) 방문 코스 선택한 부스를 방문 목록에 담기
- 4) 스탬프 미션 방문 완료 부스의 개수 표시하기
- 5) CSV 다운로드 방문 코스를 파일로 저장하기

브랜치 병합 실습

병합과 출시 확인

통합 순서

각자 실행 확인 → 커밋 → main 병합 → 전체 앱 재실행

최종 사용자 미션

실내 부스를 찾아 방문 코스를 만들고 CSV로 저장

ChatGPT 코드: 직접 실행·수정한 내용을 README에 기록

브랜치 병합 실습

예시 앱: 부스 검색

실내 부스 필터

부스 찾기

관심 분야

전체

▼

☒ 실내 부스만 보기

비가 올 때는 실내 부스만 표시할 수 있습니다.

실내 조건을 켜면 야외 부스가
검색 결과에서 제외

검색 결과

조건에 맞는 부스

5곳을 찾았습니다.

부스	분야	위치	내용
AI 포토존	체험	학생회관 1층	AI로 축제 기념사진 만들기
DIY 키링 공방	체험	학생회관 2층	나만의 키링 직접 만들기
보드게임 라운지	놀이	도서관 로비	친구와 협동 게임 즐기기
독립출판 전시	전시	도서관 1층	학생 제작 책과 잡지 살펴보기
과학 마술 교실	체험	강의동 103호	간단한 실험으로 원리 알아보기

브랜치 병합 실습

예시 앱: 추천과 방문 코스

조건에 맞는 추천

오늘의 추천

부스 한 곳 추천받기

독립출판 전시·도서관 1층

학생 제작 책과 잡지 살펴보기

추천은 현재 검색 조건 안에서 선택

선택한 코스를 CSV 파일로 저장

방문 코스와 CSV 저장

나의 방문 코스

방문할 부스

AI 포토존 × DIY 키링 공방 ×

부스	위치	구분
AI 포토존	학생회관 1층	실내
DIY 키링 공방	학생회관 2층	실내

방문 코스 CSV 다운로드

▼ 스탬프 미션

브랜치 병합 실습

병합과 출시 확인

돌발 요청: 비가 옵니다

- ① '실내 부스만 보기'를 검색과 추천에 적용
- ② 방문 코스에도 실내·야외 정보를 표시

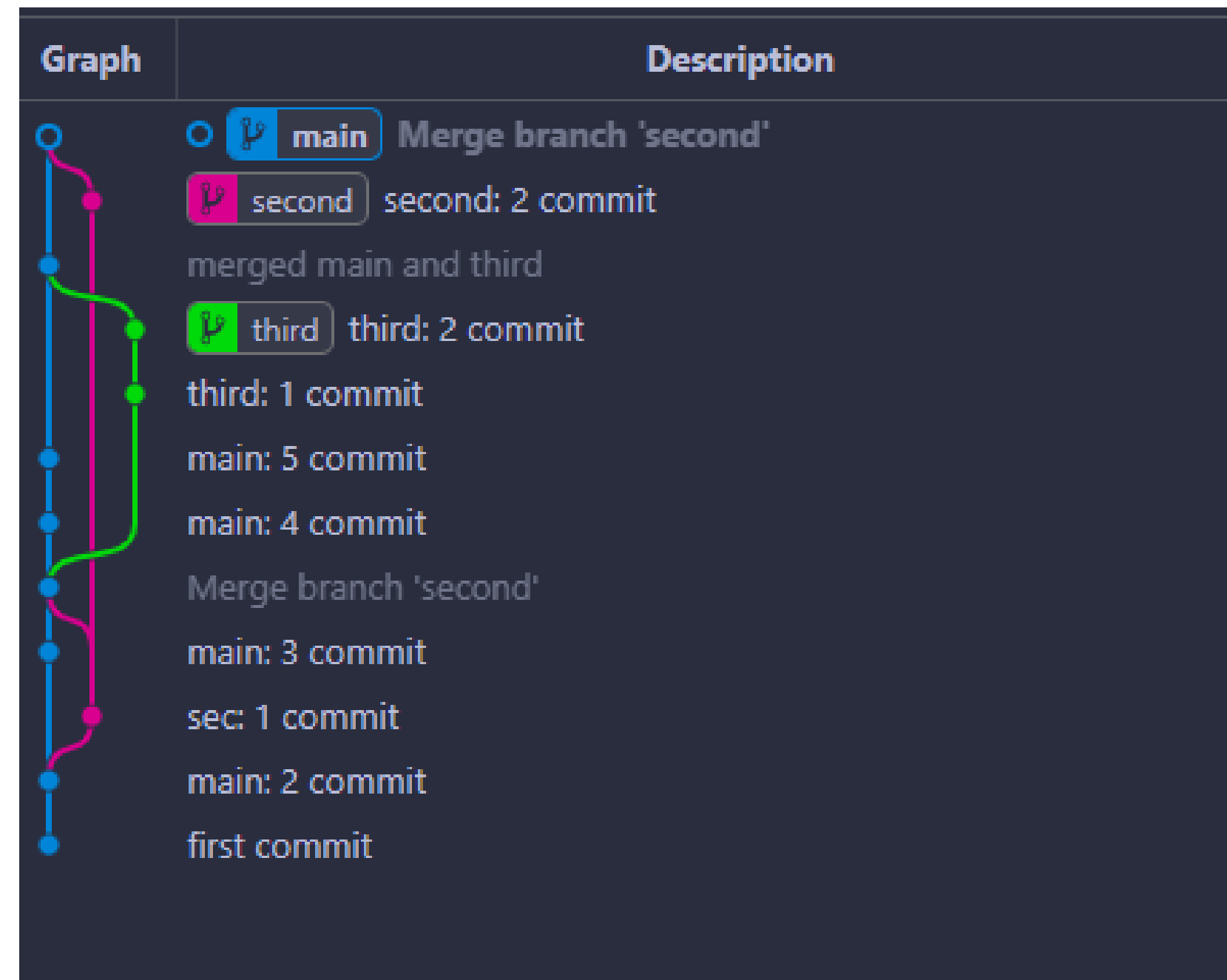
브랜치 실습

한림대학교

과제

CLASSSUM

- (1) 우측 Graph에 따라 Branch 생성 & 병합 & 삭제
- (2) Git 명령어 (text로 정리)
- (3) 적어도 1회 충돌 해결! (스크린샷)
- (4) 최종 Graph 제출 (스크린샷)



Oh My Git

❖ Git 시각화 게임

<https://ohmygit.org/>

다운로드:

<https://blinry.itch.io/oh-my-git>

▼ 오늘

oh-my-git ✓

oh-my-git.pck

▼ 지난 달

dependencies



BackReloadToggle music

Living dangerously

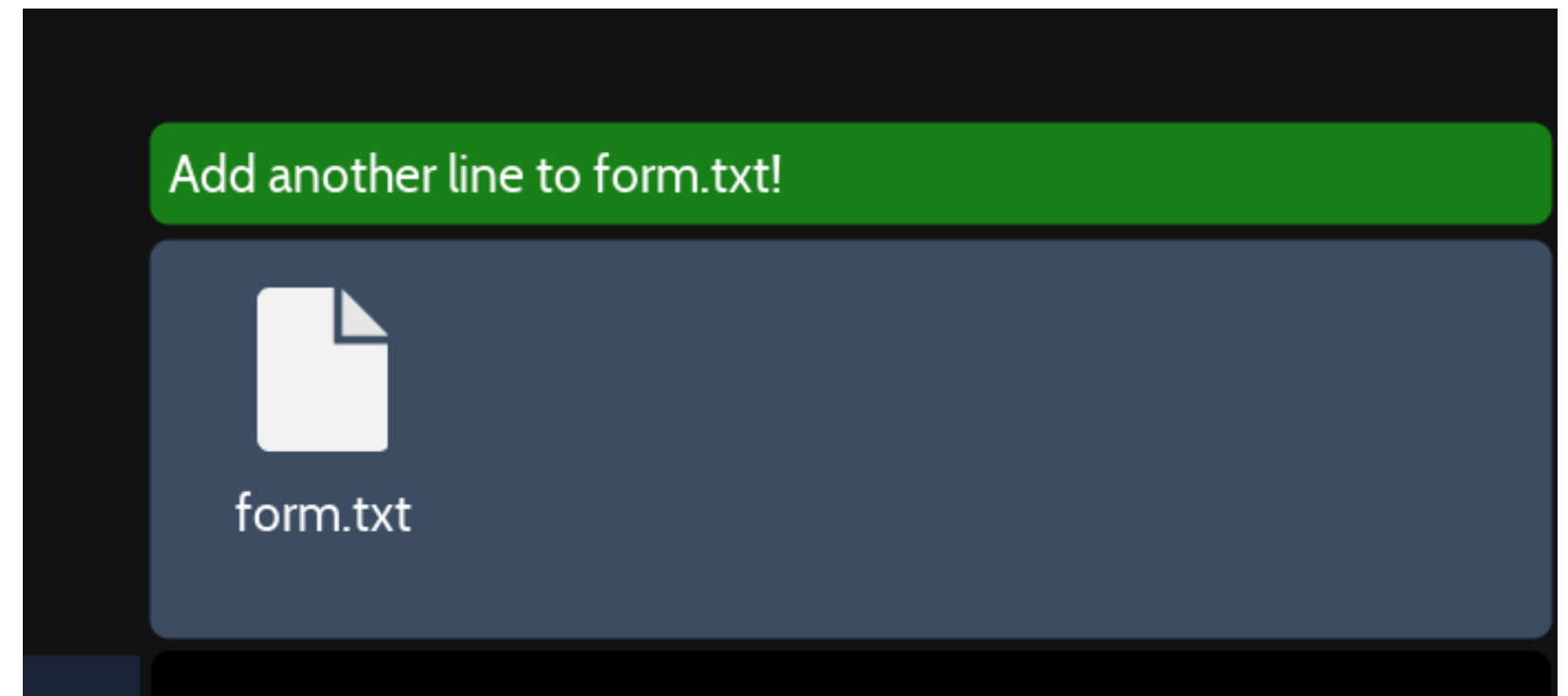
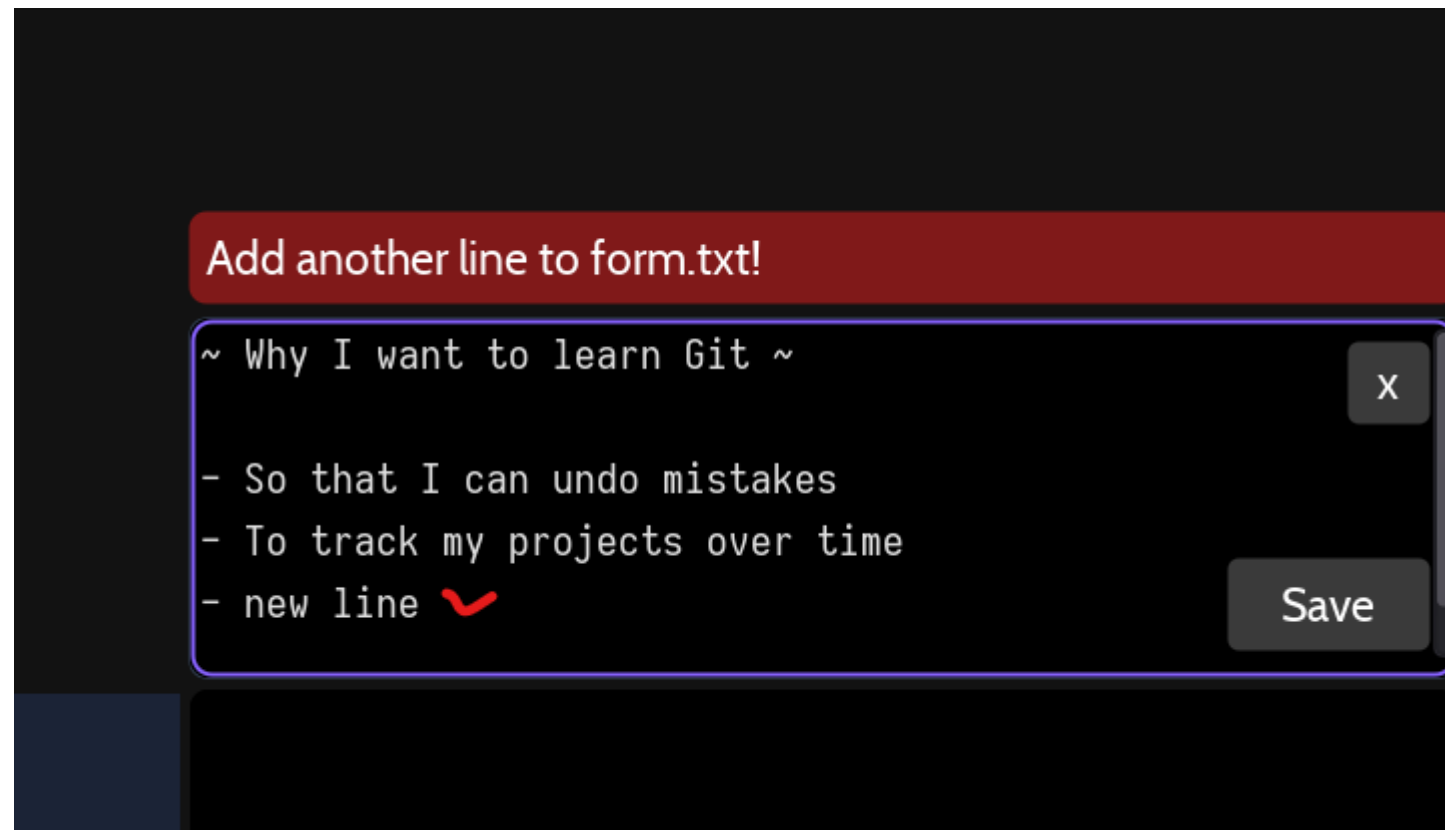
So you have decided to apply for time travel school, to learn how to use this time machine called "Git"!

How exciting!

You're almost done with the paperwork! You just need to fill in one more reason why you want to learn Git.

Add another line to form.txt!

form.txt



오늘은 여기까지.

Thank you for listening.